

Hyperdimensional Computing:

Linking symbolic logic to neural networks

Prachya Boonkwan (Arm)

School of ICT, SIIT,
Thammasat University

prachya@siit.tu.ac.th, kaamanita@gmail.com

Slides URL $\Rightarrow$ <https://tinyurl.com/yu83u2y6>



Who? Me?

- Nickname: **Arm** (P'/N'/E' Arm, etc.)
- Born: Aug 1981
- Work:
 - Researcher at NECTEC 2005-2024
 - Lecturer at SIIT, Thammasat University 2025-onw
- Education
 - B.Eng & M.Eng, CPE Kasetsart University
 - Obtained Ministry of Science Scholarship in early 2008
 - Did a PhD in Informatics (AI & Computational Linguistics) at University of Edinburgh, UK from 2008 to 2013 (4.5 years)



Outline

- Introduction
- Hyperdimensional computing (HDC)
- Encoding complex data structures
- Vector-symbolic architecture and ML
- Implementation with TorchHD
- NLP and knowledge graph via HDC
- Conclusion

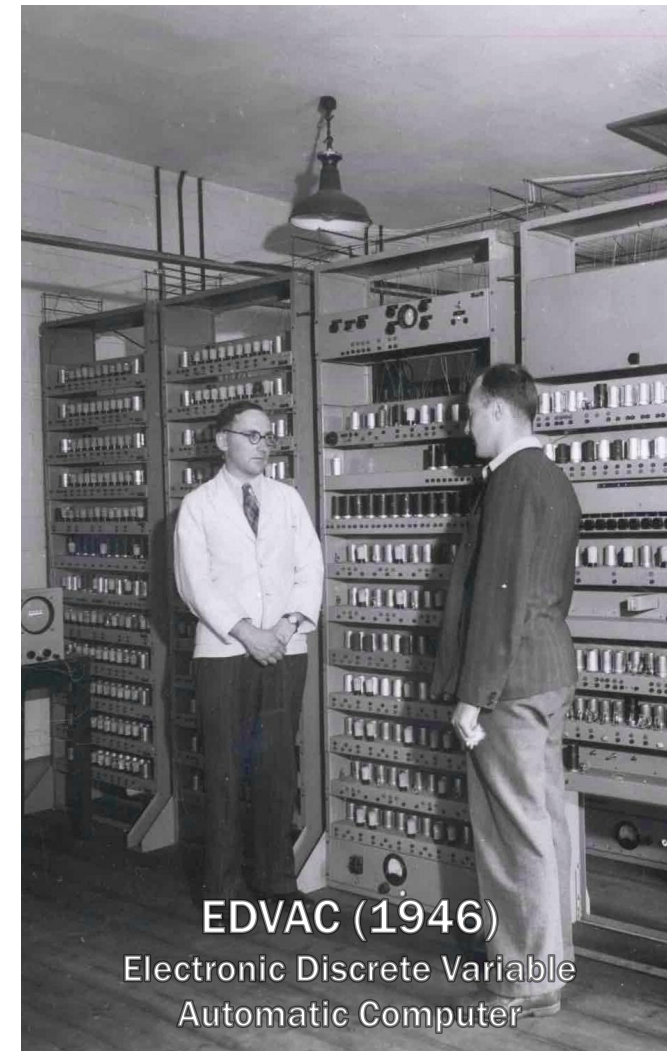
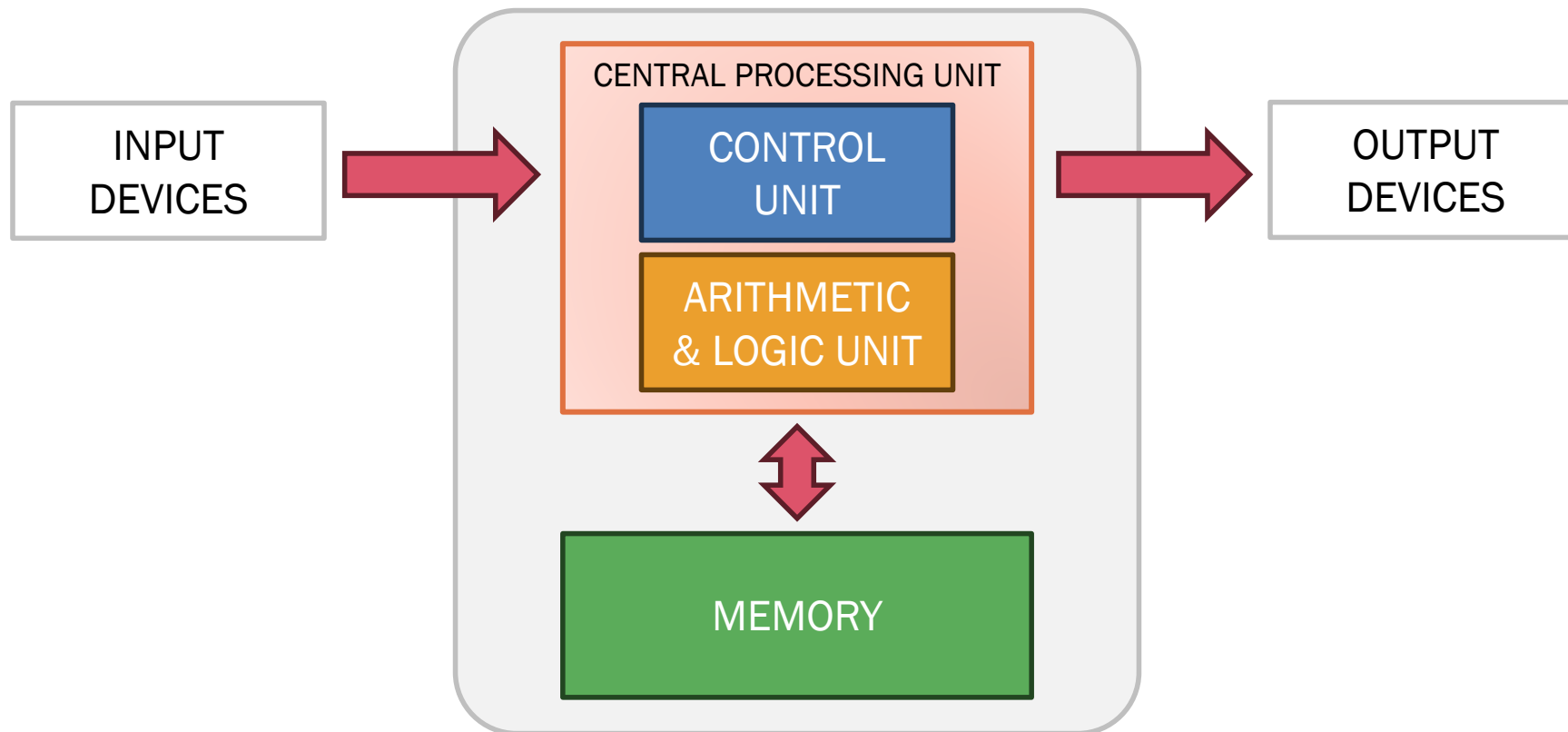


**HALLUCINATORY
CONTENTS AHEAD**

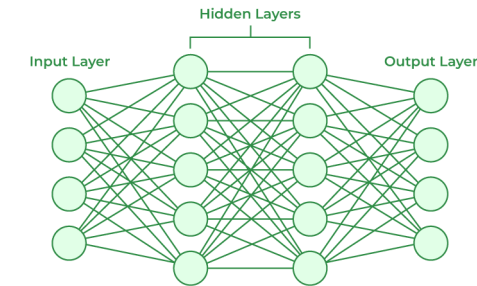
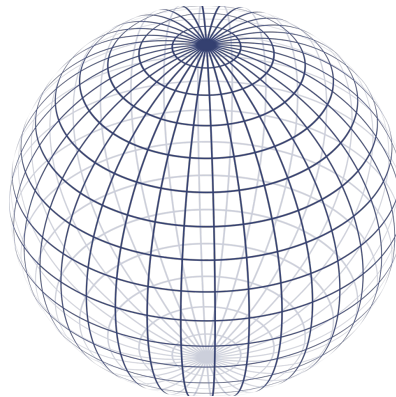
1. Introduction

von Neumann Architecture

- System design for general-purpose stored-program computers (von Neumann, 1945)



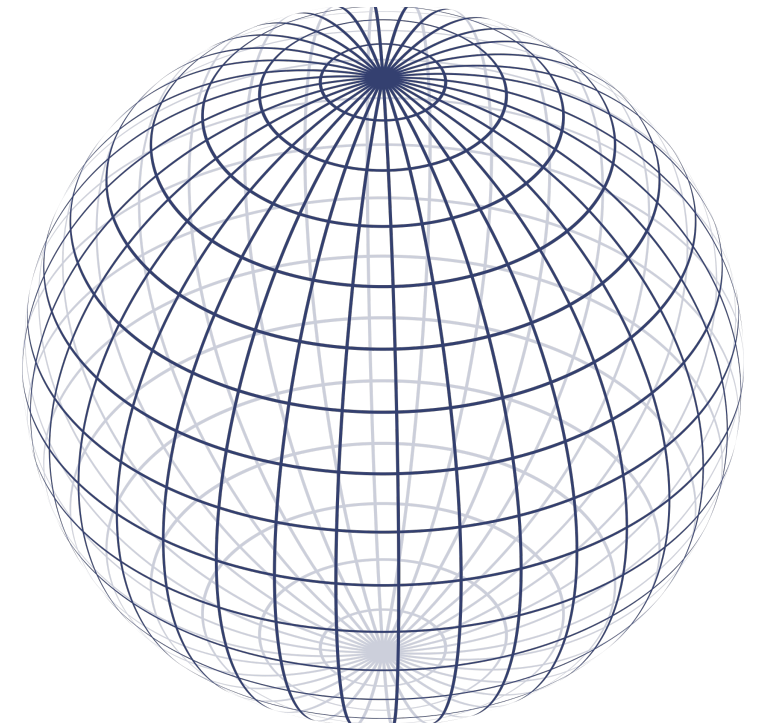
Brain v.s. von Neumann Computers



	Brain	von Neumann Computer
Representation	Non-binary symbols and logical rules	Binary 0/1
Data manipulation	Non-arithmetic operations	Arithmetic and logical operations (fast and accurate)
Inference	Learning from examples and reasoning with logical inference	Closely following a prescribed instruction set (a.k.a. 'program')
Learning representation	HDC: Symbol represented as high-dimensional vector (e.g. 10,000 dims)	ANN: Data item represented as dense low-dimensional vector (e.g. 512 dims)
Robustness	Robust and insensitive to noise due to high dimensionality	Sensitive to noise, suitable for data generation

In this talk

- Hyperdimensional computing (HDC)
 - Paradigm shift from the von Neumann Architecture
 - Bridge between symbolic logic and vector-based machine learning techniques
 - Accurate, noise-tolerant vector operations for symbolic computation and logical inference



P. Kanerva. [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#). Cognitive Computing 1:139–159. 2009.

2. Hyperdimensional Computing

Hyperdimensional Computing (HDC)

- Symbols are represented as **hyperdimensional vectors** (HVs)
 - **Noise-tolerant**: allowing errors in some of abundant dimensions
 - **Holographic**: the vectors scatter throughout the space and never approach the origin point, guaranteeing distinction [ὅλο = whole, γραφή = writing]
 - **Random**: each HV is sampled from the hyperspace, and a HDC program always works on different random initializations

$$\left. \begin{array}{l} \text{'hello'} \\ \text{'world'} \\ \text{'prachya'} \\ \text{'boonkwan'} \end{array} \right\} \mathbf{v}_{\text{symbol}} \sim \text{IID} \left(\{+1, -1\} \right)^{10000}$$

$$\underbrace{\langle +1, +1, -1, -1, +1, \dots, -1, -1, +1, -1, -1 \rangle}_{10,000 \text{ dimensions of bipolar values}}$$

Hyperdimensional Vector Algebra

- Three operations on HVs are used to form program logics

- Elementwise addition $\oplus$
(a.k.a. **bundling**)

$$\begin{array}{r}
 \langle +1 \quad -1 \quad +1 \quad +1 \quad -1 \quad +1 \quad -1 \rangle \\
 \langle +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad -1 \quad -1 \rangle \\
 \oplus \quad \langle -1 \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \rangle \\
 \hline
 \langle +1 \quad -1 \quad +3 \quad -1 \quad -3 \quad +1 \quad -1 \rangle
 \end{array}$$

- Elementwise multiplication $\otimes$
(a.k.a. **binding**)

$$\begin{array}{r}
 \langle +1 \quad -1 \quad +1 \quad +1 \quad -1 \quad +1 \quad -1 \rangle \\
 \langle +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad -1 \quad -1 \rangle \\
 \otimes \quad \langle -1 \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \rangle \\
 \hline
 \langle -1 \quad +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad +1 \rangle
 \end{array}$$

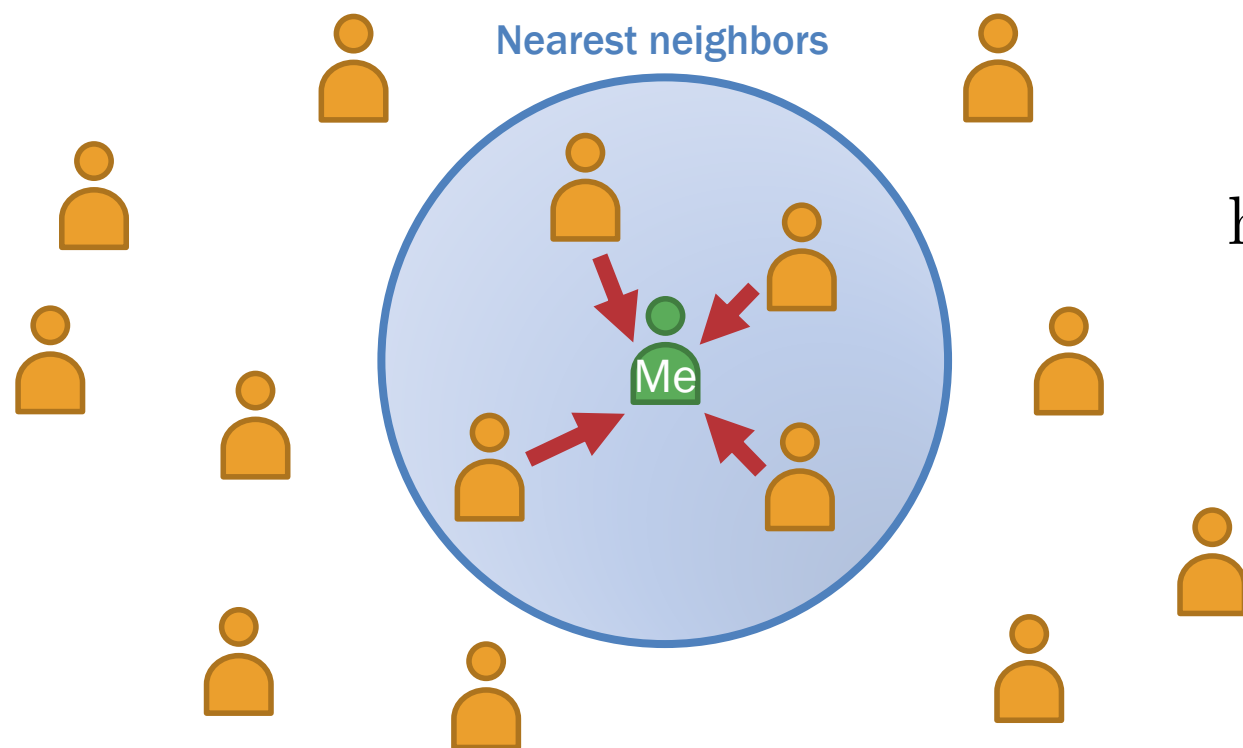
N.B. For any HV $\mathbf{x}$,
 $\mathbf{x} \otimes \mathbf{x} = \mathbf{1}$ always.

- Permutation p^N

$$\begin{array}{r}
 p^1 \quad \langle \boxed{-1} \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \quad \boxed{} \rangle \\
 \hline
 \langle \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \quad -1 \rangle
 \end{array}$$

Associative Memory

- **Cleanup:** Given a noisy HV, we find the nearest neighbor among the HVs stored in the associative memory with a similarity measure



$$\text{cossim}(\mathbf{x}, \mathbf{y}) = \arccos \frac{\mathbf{x} \cdot \mathbf{y}}{|\mathbf{x}| \times |\mathbf{y}|}$$

$$\text{hamming}(\mathbf{x}, \mathbf{y}) = \#_{\text{bit-match}}(\mathbf{x}, \mathbf{y})$$

$$\text{dotsim}(\mathbf{x}, \mathbf{y}) = \mathbf{x} \cdot \mathbf{y}$$

Each point is stored in a k -dimensional tree (KDTree) for ease of retrieval

3. Encoding Complex Data Structure

Hash Table

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 1:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

Hash Table

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 1:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$x \otimes a$

$y \otimes b$

$z \otimes c$

Hash Table

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 1:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$x \otimes a$

$y \otimes b$

$z \otimes c$

3. **Bundle** all pairs with superposition.

$$h = (x \otimes a) \oplus (y \otimes b) \oplus (z \otimes c)$$

N.B. h always stays of the same dimensions, regardless of how many entries are added to the hash table h . This is equivalent to **superposition**.

Hash Table

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 1:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$$x \otimes a$$

$$y \otimes b$$

$$z \otimes c$$

3. Bundle all pairs with superposition.

$$h = (x \otimes a) \oplus (y \otimes b) \oplus (z \otimes c)$$

4. Release a value by multiplication.

$$x \otimes (x \otimes a) = a$$

N.B. Releasing is opposite to binding

Hash Table

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 1:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$$x \otimes a$$

$$y \otimes b$$

$$z \otimes c$$

3. Bundle all pairs with superposition.

$$h = (x \otimes a) \oplus (y \otimes b) \oplus (z \otimes c)$$

4. Release a value by multiplication.

$$x \otimes (x \otimes a) = a$$

5. Release a value by multiplication.

$$\begin{aligned} x \otimes h &= [x \otimes (x \otimes a)] \oplus [x \otimes (y \otimes b)] \oplus [x \otimes (z \otimes c)] \\ &= [x \otimes (x \otimes a)] \oplus \text{noise} \oplus \text{noise} \\ &\approx x \otimes (x \otimes a) = a \end{aligned}$$

Sequence

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 2:** sequence 'hello'

1. Initialize with random vectors. $\mathbf{h}, \mathbf{e}, \mathbf{l}, \mathbf{o}$
2. Encode the sequence with permutation and binding.

$$\begin{aligned}
 |\text{hello}\rangle &= p^1|\text{hell}\rangle \otimes \mathbf{o} \\
 &= p^1(p^1|\text{hel}\rangle \otimes \mathbf{l}) \otimes \mathbf{o} \\
 &= p^1(p^1(p^1|\text{he}\rangle \otimes \mathbf{l}) \otimes \mathbf{l}) \otimes \mathbf{o} \\
 &= p^1(p^1(p^1(p^1|\mathbf{h}\rangle \otimes \mathbf{e}) \otimes \mathbf{l}) \otimes \mathbf{l}) \otimes \mathbf{o} \\
 &= p^1(p^1(p^1(p^1\mathbf{h} \otimes \mathbf{e}) \otimes \mathbf{l}) \otimes \mathbf{l}) \otimes \mathbf{o} \\
 &= p^4\mathbf{h} \otimes p^3\mathbf{e} \otimes p^2\mathbf{l} \otimes p^1\mathbf{l} \otimes \mathbf{o}
 \end{aligned}$$

Permutation preserves the order by differentiating:

- $\{w_1, w_2\} = \{w_2, w_1\}$
 $= \mathbf{w}_1 \oplus \mathbf{w}_2$
- $(w_1, w_2) = p(\mathbf{w}_1) \otimes \mathbf{w}_2$
- $(w_2, w_1) = p(\mathbf{w}_2) \otimes \mathbf{w}_1$

Knowledge Graph

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- **Example 3:** predicate $\text{ancestor}(x, y)$

1. Decompose the predicate according to their arguments. $\mathbf{anc}_1, \mathbf{anc}_2$

2. Initialize all symbols with random vectors. $\mathbf{anc}_1, \mathbf{anc}_2$
 $\mathbf{john}, \mathbf{mary}, \mathbf{betty}, \mathbf{peter}$

3. Bind the decomposed predicate to each argument.

$$\text{ancestor}(\mathbf{john}, \mathbf{mary}) \Rightarrow (\mathbf{anc}_1 \otimes \mathbf{john}) \otimes p^1(\mathbf{anc}_2 \otimes \mathbf{mary})$$

$$\text{ancestor}(\mathbf{mary}, \mathbf{betty}) \Rightarrow (\mathbf{anc}_1 \otimes \mathbf{mary}) \otimes p^1(\mathbf{anc}_2 \otimes \mathbf{betty})$$

$$\text{ancestor}(\mathbf{betty}, \mathbf{peter}) \Rightarrow (\mathbf{anc}_1 \otimes \mathbf{betty}) \otimes p^1(\mathbf{anc}_2 \otimes \mathbf{peter})$$

4. Bundle all predicates. $\mathbf{G} = [(\mathbf{anc}_1 \otimes \mathbf{john}) \otimes p^1(\mathbf{anc}_2 \otimes \mathbf{mary})]$
 $\oplus [(\mathbf{anc}_1 \otimes \mathbf{mary}) \otimes p^1(\mathbf{anc}_2 \otimes \mathbf{betty})]$
 $\oplus [(\mathbf{anc}_1 \otimes \mathbf{betty}) \otimes p^1(\mathbf{anc}_2 \otimes \mathbf{peter})]$

Image

- **Example 4:** RGB image

1. Initialize positions and RGB values with random vectors with
 locality-preserving functions.

$$\mathbf{x}_1, \dots, \mathbf{x}_{640} \quad \mathbf{r}_0, \dots, \mathbf{r}_{255}$$

$$\mathbf{y}_1, \dots, \mathbf{y}_{480} \quad \mathbf{g}_0, \dots, \mathbf{g}_{255}$$

$$\mathbf{b}_0, \dots, \mathbf{b}_{255}$$

2. Bind each pixel (r, g, b) with multiplication.

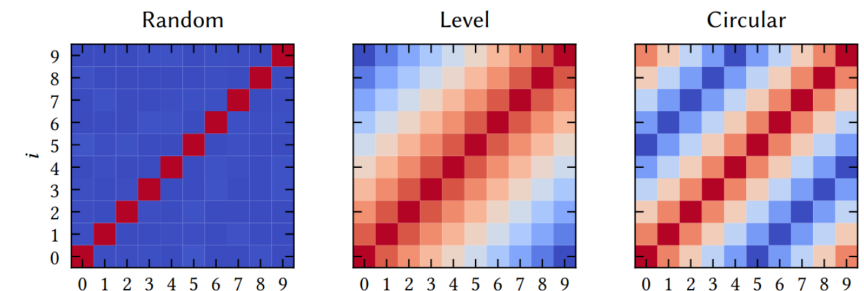
$$\mathbf{p}(r, g, b) = p^3 \mathbf{r}_r \otimes p^2 \mathbf{g}_g \otimes p^1 \mathbf{b}_b$$

3. Bundle all pixels and their positions.

$$\mathbf{I} = \bigoplus_{x=1}^{640} \bigoplus_{y=1}^{480} [(\mathbf{x}_x \otimes \mathbf{y}_y) \otimes \mathbf{p}(r, g, b)]$$

(elementwise summation: Σ)

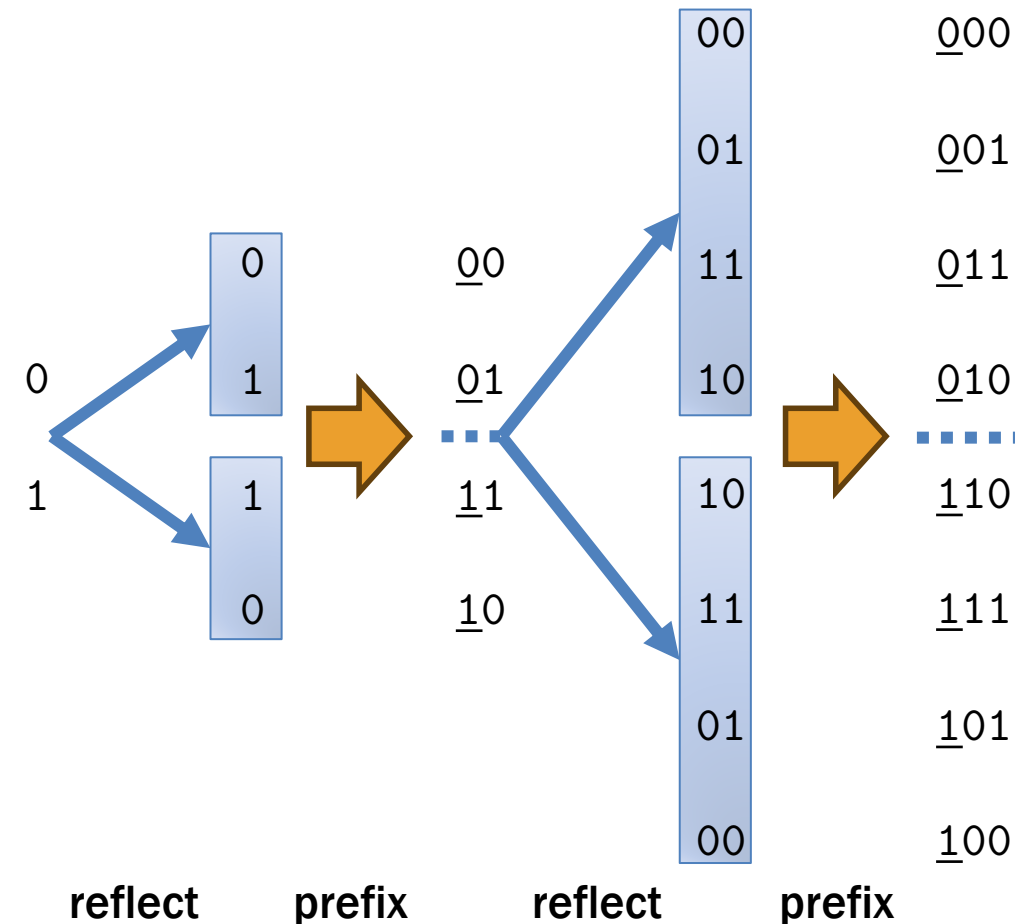
L. Ge and K. K. Parhi, [Classification using Hyperdimensional Computing: A Review](#), IEEE Circuits and Systems Magazine. 2020.



Locality-preserving random vectors

Linear Order in Random Vectors

- Locality-preserving random vectors with linear order
- **Grey code**
 - Given a sequence of binary codes
 - Reflect: mirror the sequence
 - Prefix:
 - Add 0 to the original sequence
 - Add 1 to the mirrored sequence
 - Carry on the reflect-and-prefix loop



Long Text

J. Karlgren and P. Kanerva. [High-dimensional distributed semantic spaces for utterances](#). Natural Language Engineering 25, no. 4 (2019): 503-517.

- **Example 5:** long document

1. Initialize characters with random vectors.

a b c d e x y z #

2. Each subword is encoded with permutation.

'mis-under-stand-ing'

$$|\text{mis}\#\rangle = p^3 \mathbf{m} \otimes p^2 \mathbf{i} \otimes p^1 \mathbf{s} \otimes \#$$

$$|\#\text{under}\#\rangle = p^6 \# \otimes p^5 \mathbf{u} \otimes p^4 \mathbf{n} \otimes p^3 \mathbf{d} \otimes p^2 \mathbf{e} \otimes p^1 \mathbf{r} \otimes \#$$

$$|\#\text{stand}\#\rangle = p^6 \# \otimes p^5 \mathbf{s} \otimes p^4 \mathbf{t} \otimes p^3 \mathbf{a} \otimes p^2 \mathbf{n} \otimes p^1 \mathbf{d} \otimes \#$$

$$|\#\text{ing}\rangle = p^3 \# \otimes p^2 \mathbf{i} \otimes p^1 \mathbf{n} \otimes \mathbf{g}$$

3. Each n -gram of subwords is encoded with permutation.

$$|-\text{under-stand-ing}\rangle \quad [\text{trigram}]$$

$$= p^2 |\#\text{under}\#\rangle \otimes p^1 |\#\text{stand}\#\rangle \otimes |\#\text{ing}\rangle$$

4. Bundle all subword n -grams.

$$|\text{com-plete mis-under-stand-ing}\rangle$$

$$= |\text{com-plete mis}\rangle$$

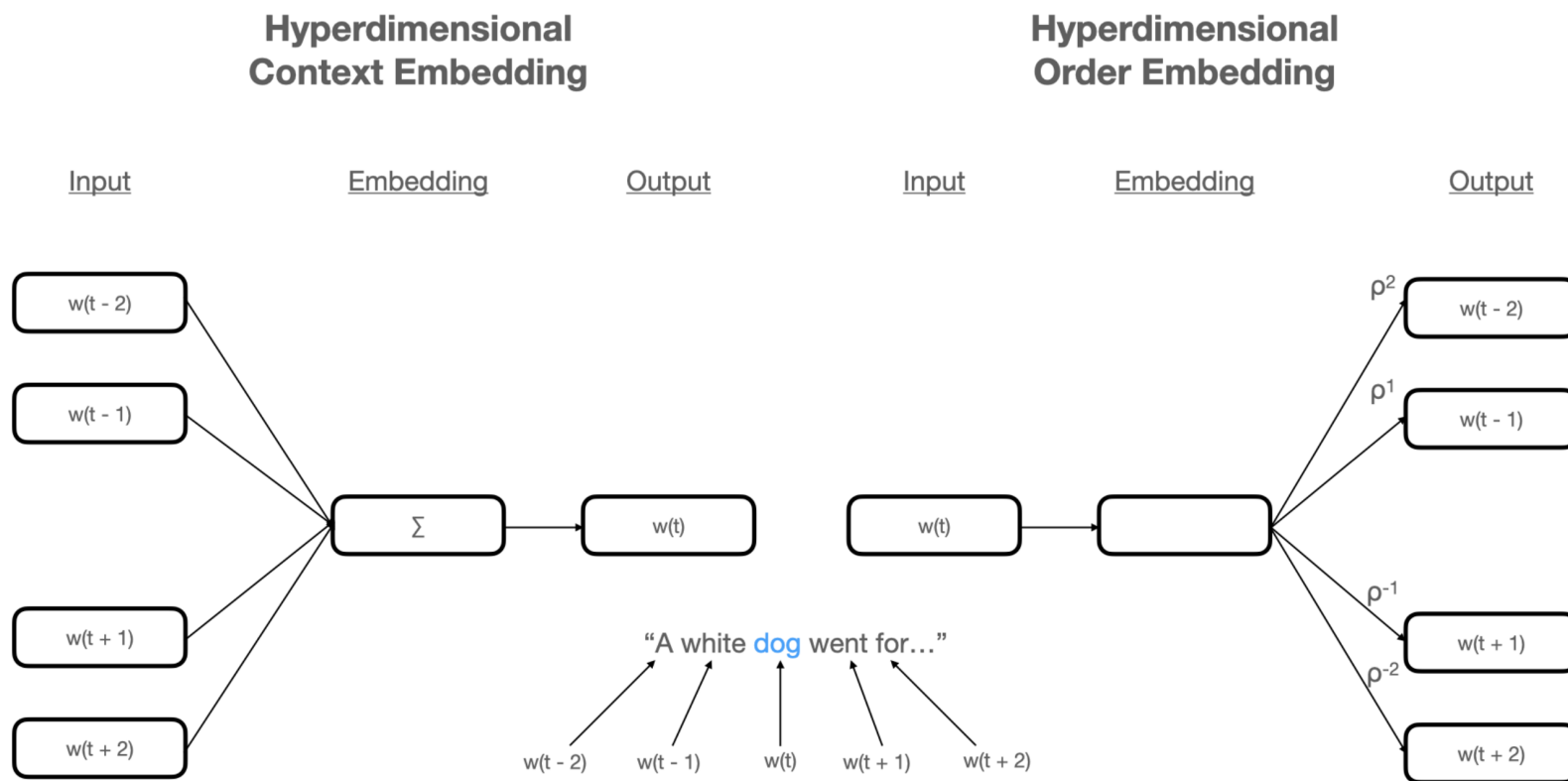
$$\oplus |\text{plete mis-under}\rangle$$

$$\oplus |\text{mis-under-stand}\rangle$$

$$\oplus |-\text{under-stand-ing}\rangle$$

Language Models

R. Moughan, [Parallel Architectures for Hyperdimensional Computing](#), Technical Report No. UCB/EECS-2021-67. UC Berkeley. 2021.



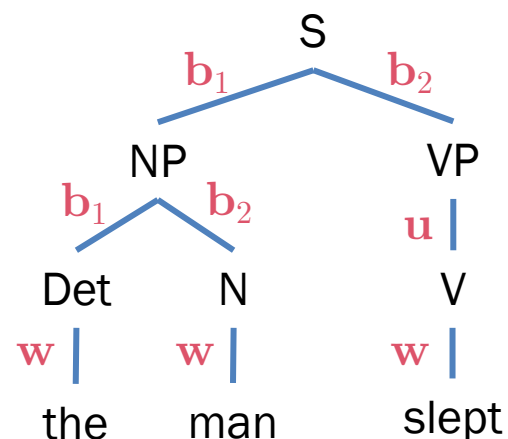
- Words are encoded as a HV of context words
- Instead of using word HV, we employ contextual HV

$$\begin{aligned}
 |\text{dog}\rangle_{\text{ctx}} = & p^2 |a\rangle_{\text{word}} \\
 & \oplus p^1 |\text{white}\rangle_{\text{word}} \\
 & \oplus p^{-1} |\text{went}\rangle_{\text{word}} \\
 & \oplus p^{-2} |\text{for}\rangle_{\text{word}}
 \end{aligned}$$

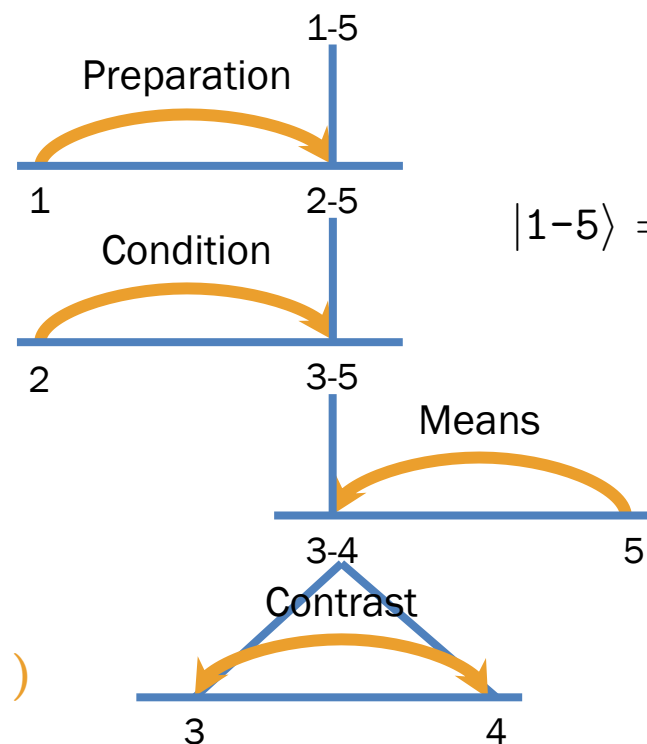
Linguistic Structures

E. P. Frady and S. J. Kent, [Resonator networks for factoring distributed representations of data structures](#), Neural Computation, 2020.

- **Example 6:** syntactic tree and discourse structure



$$\mathbf{t} = |\mathbf{S}\rangle \otimes p(\mathbf{b}_1 \otimes |\mathbf{NP}\rangle \otimes p(\mathbf{b}_1 \otimes p(|\mathbf{Det}\rangle \otimes p(\mathbf{w} \otimes |\mathbf{the}\rangle)) \oplus \mathbf{b}_2 \otimes p(|\mathbf{N}\rangle \otimes p(\mathbf{w} \otimes |\mathbf{man}\rangle)) \oplus \mathbf{b}_2 \otimes |\mathbf{VP}\rangle \otimes p(\mathbf{u} \otimes |\mathbf{V}\rangle \otimes p(\mathbf{w} \otimes |\mathbf{slept}\rangle)))$$



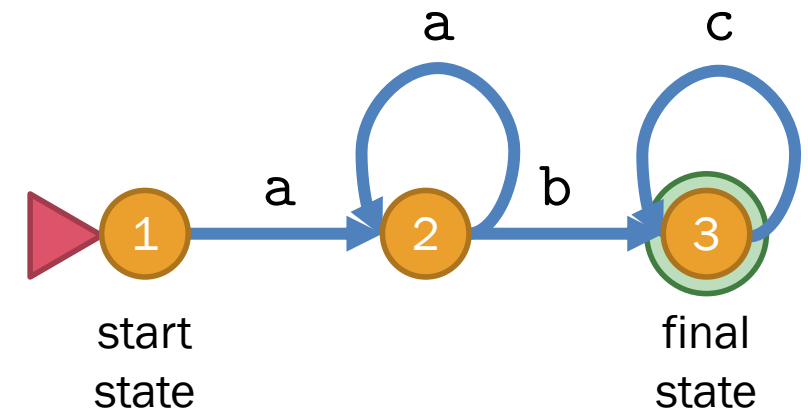
$$|1-5\rangle = |\mathbf{Prep-H}\rangle \otimes p(|\mathbf{Cond-H}\rangle \otimes p(|\mathbf{Means-H}\rangle \otimes p(|\mathbf{Ctrst-1}\rangle \otimes p|S_3\rangle \oplus |\mathbf{Ctrst-2}\rangle \otimes p|S_4\rangle \oplus |\mathbf{Means-D}\rangle \otimes p|S_5\rangle \oplus |\mathbf{Cond-D}\rangle \otimes p|S_2\rangle \oplus |\mathbf{Prep-D}\rangle \otimes p|S_1\rangle))$$

Finite-State Automata

P. Kanerva, [Hyperdimensional Computing: An Introduction to Computing in Distributed Representation with High-Dimensional Random Vectors](#), Cognitive Computing. 2009.

- Example 7:** state transitions of FSA

1. Initialize all states and symbols with random vectors
- | | | |
|--------------|--------------|--------------|
| s_1 | s_2 | s_3 |
| $\mathbf{a}$ | $\mathbf{b}$ | $\mathbf{c}$ |



2. Each state transition from s_i to s_j w.r.t. input symbol a is encoded with binding and permutation

$$|(s_i, a, s_j)\rangle = (\mathbf{s}_i \otimes \mathbf{a}) \otimes p(\mathbf{s}_j)$$

3. FSA is the binding of all state transitions

$$\mathbf{m} = \bigoplus_{k=1}^K |(s_{ki}, a_k, s_{kj})\rangle$$

4. State transition can be done by releasing

$$\mathbf{s}_j = p^{-1}(\mathbf{s}_i \otimes \mathbf{a} \otimes \mathbf{m})$$

How many dimensions do we need?

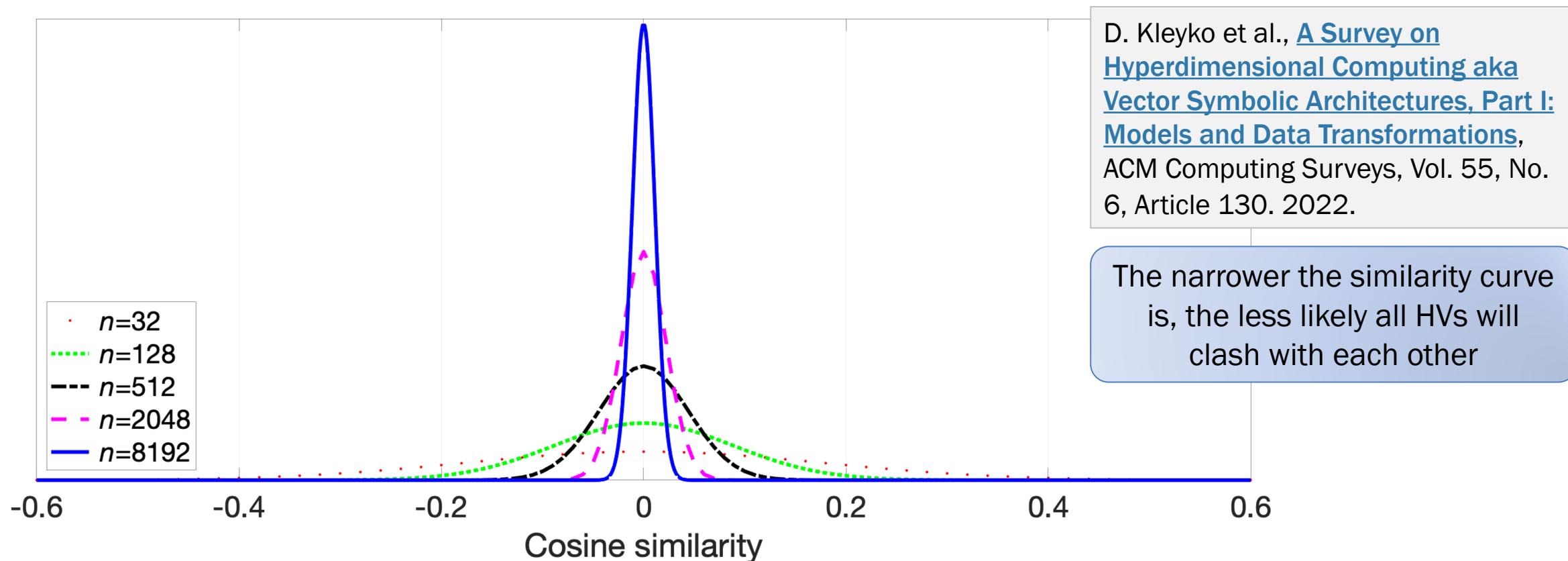
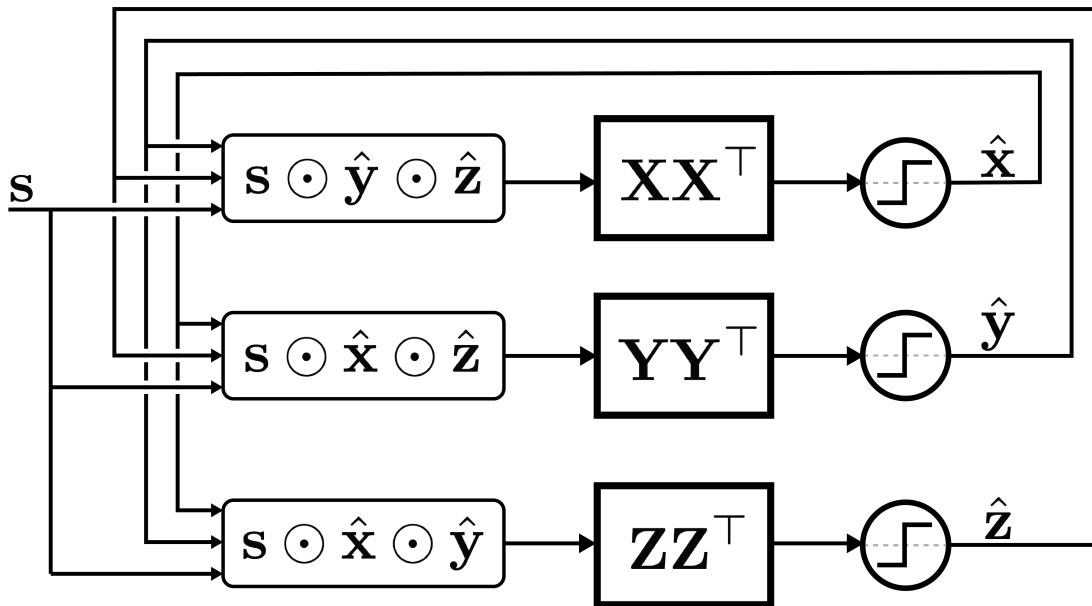


Fig. 1. Concentration of measure phenomenon between bipolar random HVs. Lines correspond to probability density functions of cosine similarities for different dimensionalities. Normal distributions were fitted to pairwise similarities for 2,000 random HVs.

Decoding Complex Data Structures

- Search $s = \mathbf{x}_i \otimes \mathbf{y}_j \otimes \mathbf{z}_k$ via **denoising resonator networks**



$$\hat{\mathbf{x}}(t+1) = g(\mathbf{X}\mathbf{X}^\top (\mathbf{s} \odot \hat{\mathbf{y}}(t) \odot \hat{\mathbf{z}}(t)))$$

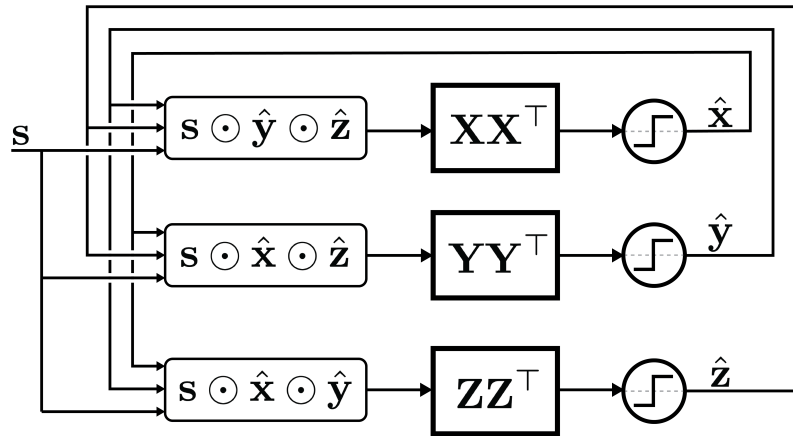
$$\hat{\mathbf{y}}(t+1) = g(\mathbf{Y}\mathbf{Y}^\top (\mathbf{s} \odot \hat{\mathbf{x}}(t) \odot \hat{\mathbf{z}}(t)))$$

$$\hat{\mathbf{z}}(t+1) = g(\mathbf{Z}\mathbf{Z}^\top (\mathbf{s} \odot \hat{\mathbf{x}}(t) \odot \hat{\mathbf{y}}(t)))$$

where:

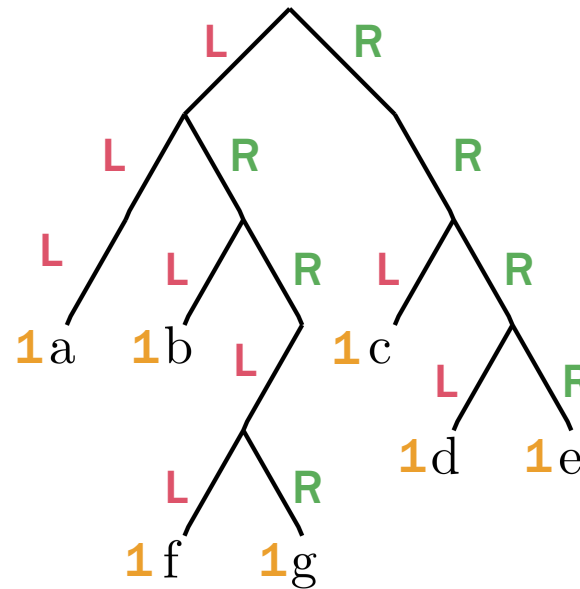
- Initially, $\hat{\mathbf{x}}(0) = \sum_i^D \mathbf{x}_i$, $\hat{\mathbf{y}}(0) = \sum_j^D \mathbf{y}_j$, etc.
- $\mathbf{X}^\top \mathbf{x}'$ = a vector of similarity scores between $\mathbf{x}'$ and each $\mathbf{x}_i$
- $\mathbf{X}(\mathbf{X}^\top \mathbf{x}')$ = weighted average of all $\mathbf{x}_i$
- $g(\cdot)$ is a threshold function in $[-1,1]$

Resonator Network



Finding a path to a node via the resonator network

1. Path is a sequence of **Left/Right/Stop** labels (**X**)
2. Given a node **n**, we will find its path by factorizing $\mathbf{t} \otimes \mathbf{n}$
3. We factorize $\mathbf{t} \otimes \mathbf{n}$ into five components: $\mathbf{X}$, $p(\mathbf{X})$, $p^2(\mathbf{X})$, $p^3(\mathbf{X})$, and $p^4(\mathbf{X})$



$$\begin{aligned} \mathbf{t} = & \mathbf{a} \otimes (\mathbf{L} \otimes p(\mathbf{L}) \otimes p^2(\mathbf{L})) \\ & \oplus \mathbf{b} \otimes (\mathbf{L} \otimes p(\mathbf{R}) \otimes p^2(\mathbf{L})) \\ & \oplus \mathbf{f} \otimes (\mathbf{L} \otimes p(\mathbf{R}) \otimes p^2(\mathbf{R}) \\ & \quad \otimes p^3(\mathbf{L}) \otimes p^4(\mathbf{L})) \\ & \oplus \dots \end{aligned}$$

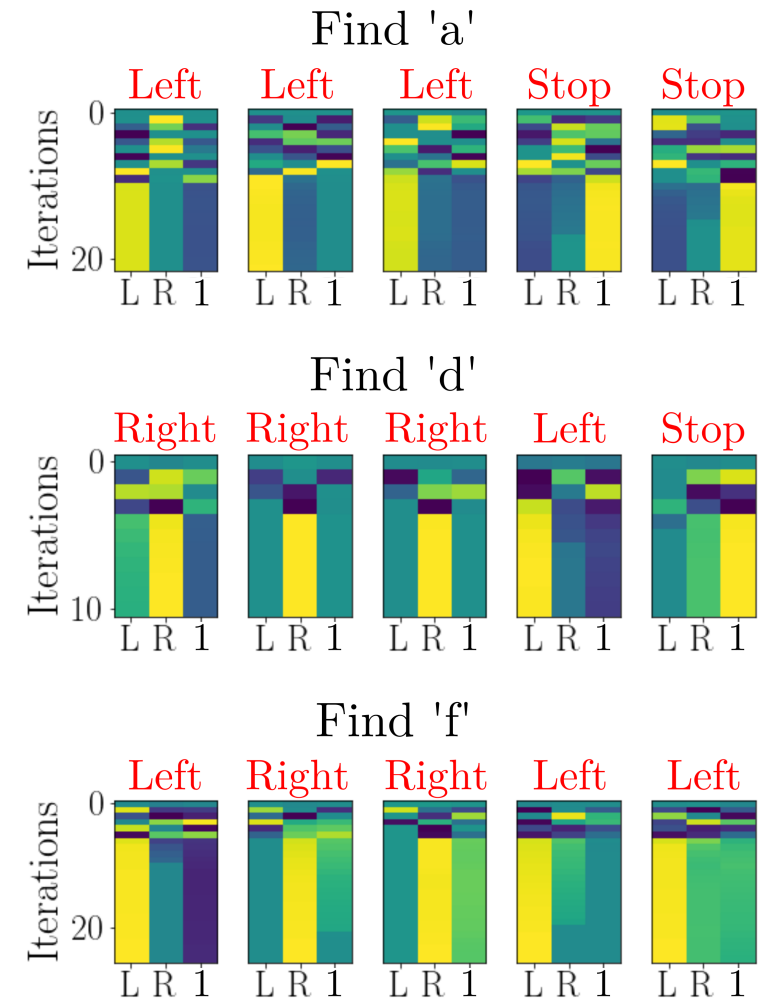


Figure 2: Tree search with a resonator network. The query of the vector **tree** produces an encoding of position which the resonator network can factor. The colored plots indicate the time evolution of $\hat{\mathbf{x}}^{(0)} \dots \hat{\mathbf{x}}^{(4)}$ (from left to right), showing the cosine similarity of each estimate to each of the three possible vectors $\rho^d(\mathbf{left})$, $\rho^d(\mathbf{right})$, $\mathbf{1}$. Purple indicates low similarity, and yellow indicates high similarity. Initially the similarity changes significantly, until the three estimators find a coherent factorization and quickly converge. Red letters indicate the converged result for each $\hat{\mathbf{x}}^{(0)} \dots \hat{\mathbf{x}}^{(4)}$.

Time Series Analysis

K. Schlegel, P. Neubert and P. Protzel, [HDC-MiniROCKET: Explicit Time Encoding in Time Series Classification with Hyperdimensional Computing](#), International Joint Conference on Neural Networks (IJCNN), pp. 1-8. 2022.

- Given a vector input signals $\mathbf{x}$ of length T
 - Pre-compute the temporal embedding $\mathbf{p}_t$ for each t -th point with locality-preserving functions
 - Compute a cross-correlation vector between $\mathbf{x}$ and a predefined filter with dilated convolution

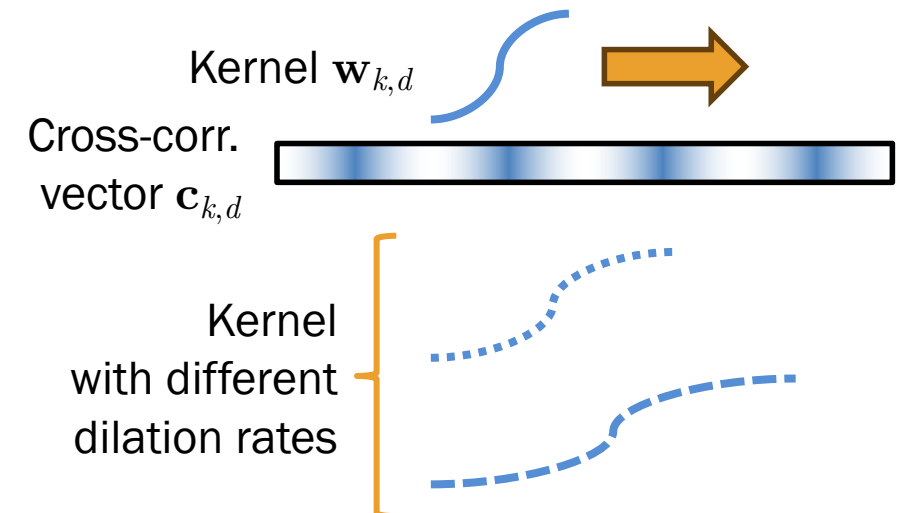
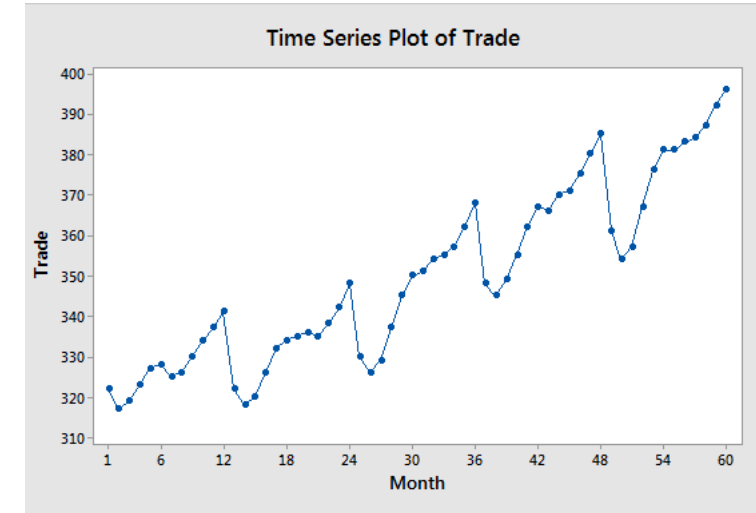
$$\mathbf{c}_{k,d} = \mathbf{X} * \mathbf{w}_{k,d}$$

where each $\mathbf{w}_k$ is a kernel vector (filter), and d is a dilation rate (pixel skipping)

- Compute the filter-response vector w.r.t. bias b

$$\mathbf{c}'_{k,d,b}[t] = \begin{cases} \mathbf{p}_t & \mathbf{c}_{k,d}[t] > b \\ -\mathbf{p}_t & \text{otherwise} \end{cases}$$

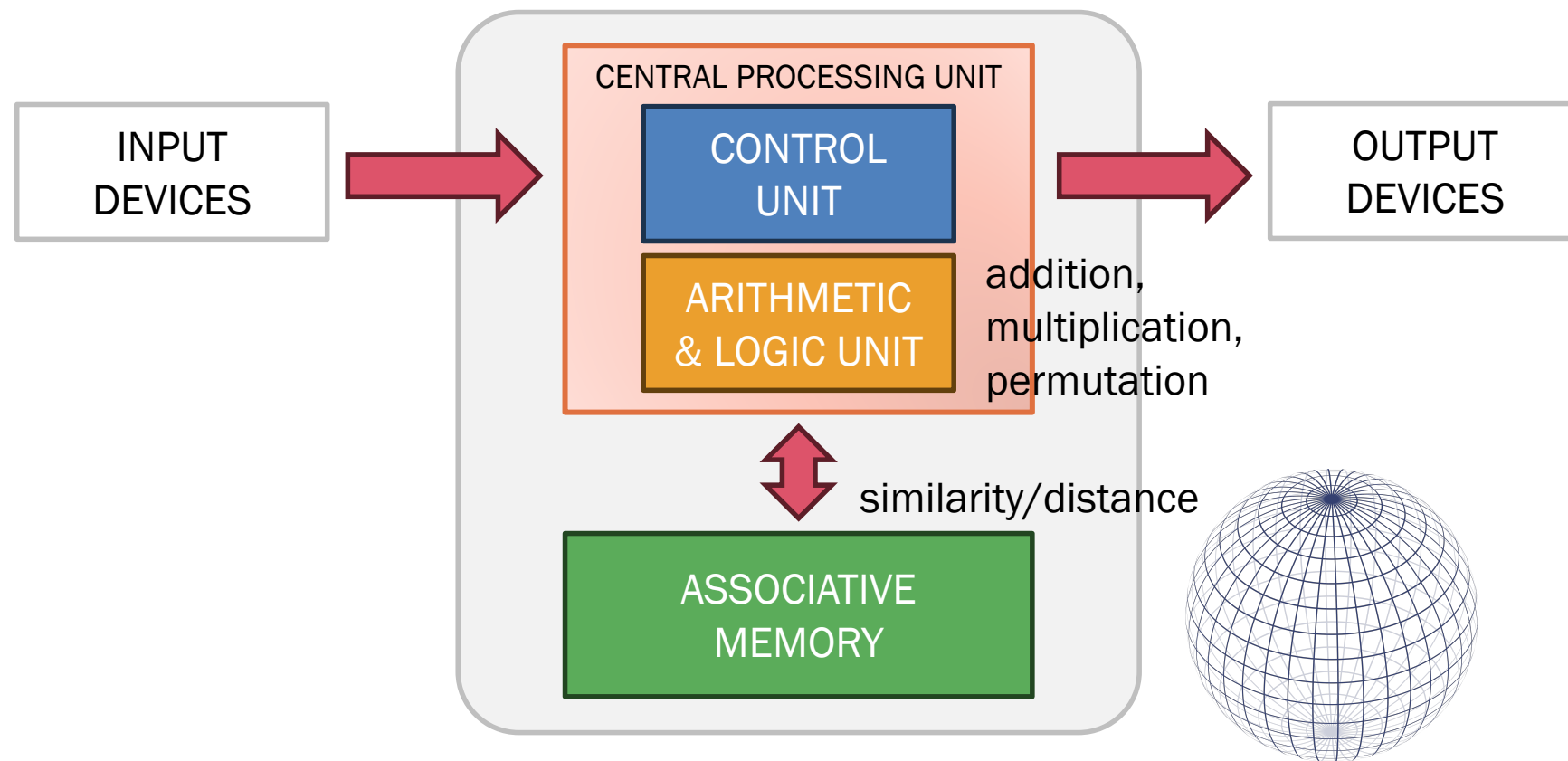
- Time series representation is: $\mathbf{s} = \bigoplus_{t=1}^T \mathbf{c}'_{k,d,b}[t]$



4. Vector-Symbolic Architecture and ML

Vector-Symbolic Architecture (VSA)

- **Non-von Neumann:** HDC-based system design for general-purpose stored-program computers

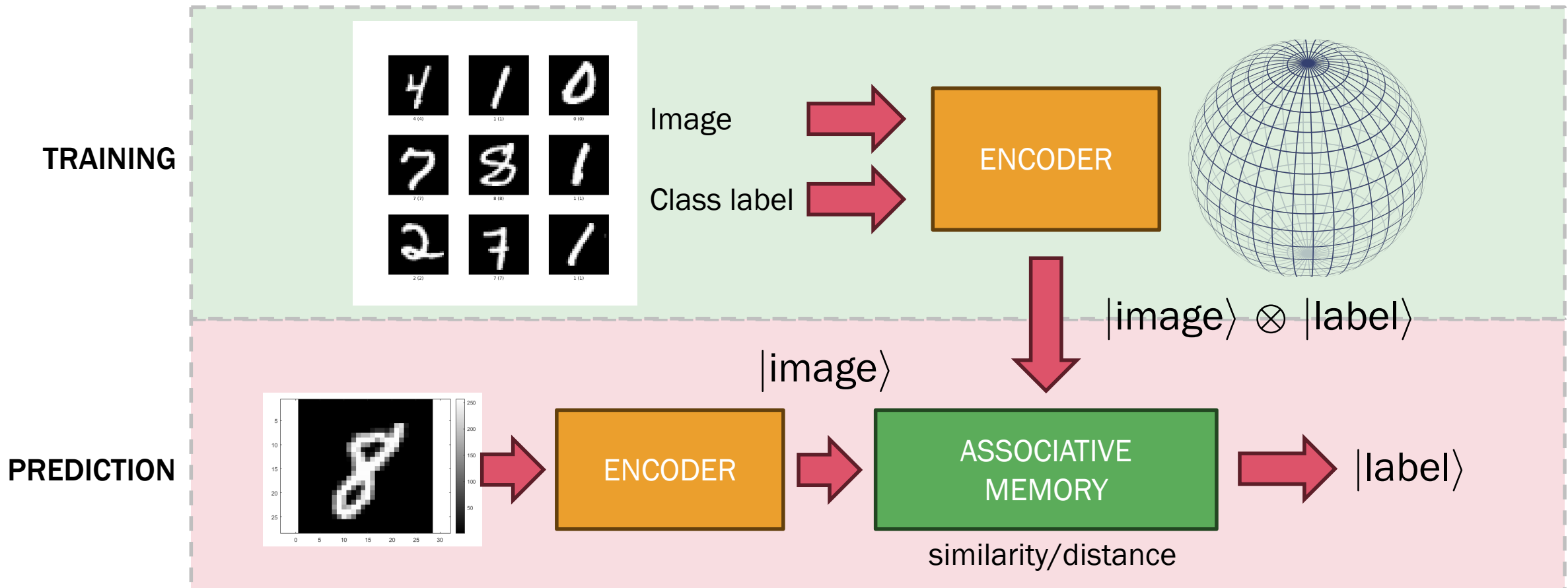


- Data structure encoding is application-oriented
 - It depends on how we get access to each component of the data structure
 - Permutation and binding are usually safe in HDC
 - Bundling may introduce noise when releasing a value — use it cautiously

Single-Pass Machine Learning with VSA

- HDC encoding allows us to perform a learning task with little to no parameter estimation

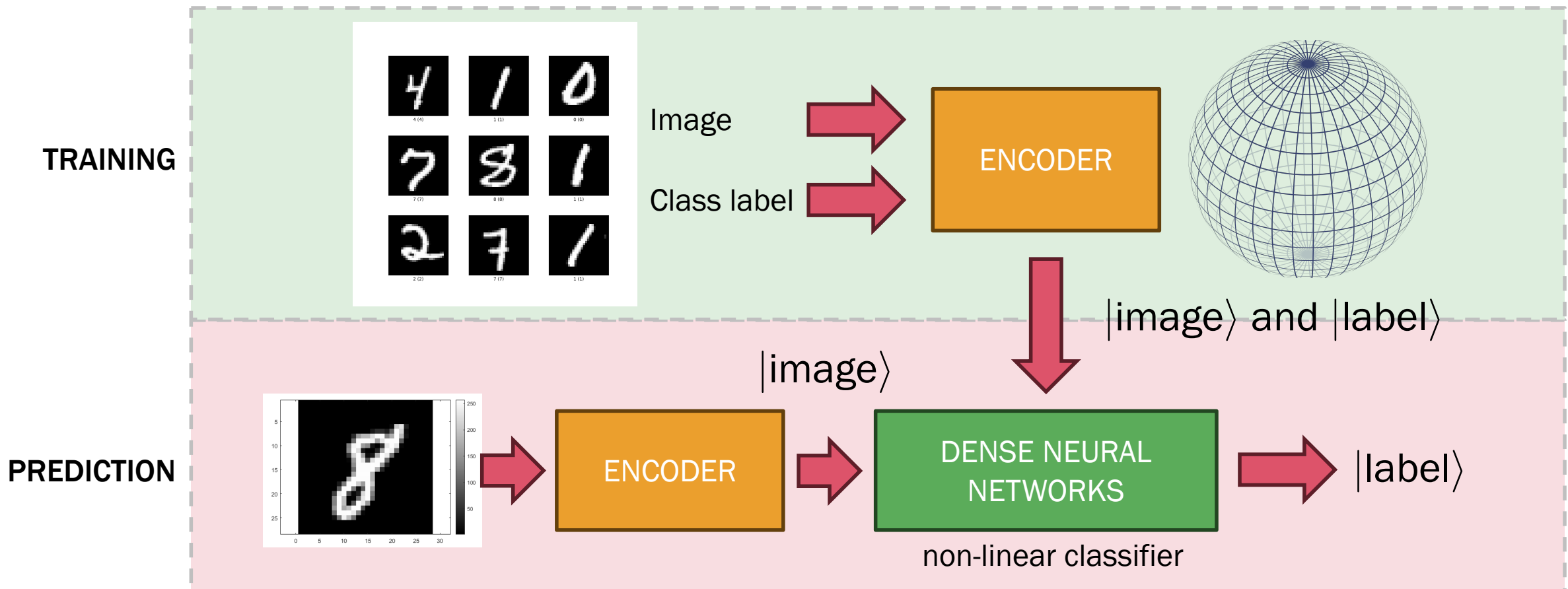
L. Ge and K. K. Parhi, [Classification using Hyperdimensional Computing: A Review](#), IEEE Circuits and Systems Magazine. 2020.



Neuro-Symbolic Learning

L. Ge and K. K. Parhi, [Classification using Hyperdimensional Computing: A Review](#), IEEE Circuits and Systems Magazine. 2020.

- Similarity-based associative memory can be replaced by trainable neural networks for flexibility



Type 1: Multiply-Add-Permute (MAP)

R. W. Gayler, [Multiplicative Binding, Representation Operators & Analogy](#), Advances in Analogy Research, 1998.

- **Commutative:** the order can change ($\mathbf{x} \otimes \mathbf{y} = \mathbf{y} \otimes \mathbf{x}$)

- **Elementwise addition** $\oplus$

$$\begin{array}{r}
 \langle +1 \quad -1 \quad +1 \quad +1 \quad -1 \quad +1 \quad -1 \rangle \\
 \langle +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad -1 \quad -1 \rangle \\
 \oplus \quad \langle -1 \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \rangle \\
 \hline
 \langle +1 \quad -1 \quad +3 \quad -1 \quad -3 \quad +1 \quad -1 \rangle
 \end{array}$$

- **Elementwise multiplication** $\otimes$

$$\begin{array}{r}
 \langle +1 \quad -1 \quad +1 \quad +1 \quad -1 \quad +1 \quad -1 \rangle \\
 \langle +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad -1 \quad -1 \rangle \\
 \otimes \quad \langle -1 \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \rangle \\
 \hline
 \langle -1 \quad +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad +1 \rangle
 \end{array}$$

N.B. For any HV $\mathbf{x}$,
 $\mathbf{x} \otimes \mathbf{x} = 1$ always.

- **Permutation** p^N

$$\begin{array}{r}
 p^1 \quad \langle \boxed{-1} \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \quad \boxed{} \rangle \\
 \hline
 \langle \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \quad -1 \rangle
 \end{array}$$

Encoding a Hash Table with MAP

- **Example 1:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$$x \otimes a$$

$$y \otimes b$$

$$z \otimes c$$

N.B. Releasing is commutative

3. Bundle all pairs with superposition.

$$h = (x \otimes a) \oplus (y \otimes b) \oplus (z \otimes c)$$

4. Release a value by multiplication.

$$x \otimes (x \otimes a) = a \quad (x \otimes a) \otimes x = a$$

5. Release a value by multiplication.

commutative (swappable)

$$\begin{aligned} x \otimes h &= [x \otimes (x \otimes a)] \oplus [x \otimes (y \otimes b)] \oplus [x \otimes (z \otimes c)] \\ &= [x \otimes (x \otimes a)] \oplus \text{noise} \oplus \text{noise} \\ &\approx x \otimes (x \otimes a) = a \end{aligned}$$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

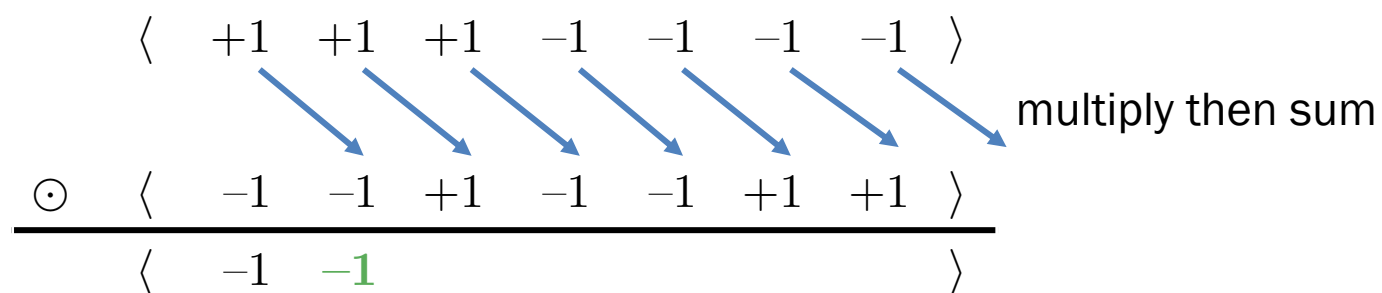
- **Circular correlation** $\odot$

$$\begin{array}{ccccccc}
 & \langle & +1 & +1 & +1 & -1 & -1 & -1 & -1 & \rangle \\
 & & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & \downarrow & & \\
 \odot & \langle & -1 & -1 & +1 & -1 & -1 & +1 & +1 & \rangle \\
 \hline
 & \langle & -1 & & & & & & & \rangle
 \end{array}
 \quad \text{multiply then sum}$$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

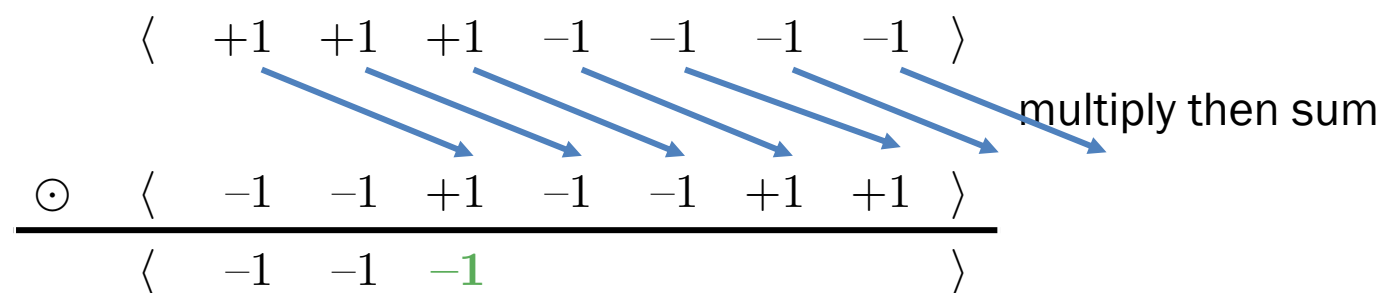
- **Circular correlation** $\odot$



Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

- **Circular correlation** $\odot$



Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

- **Circular correlation** 

[illegible]

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

- **Circular correlation** 

$\langle +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad -1 \quad -1 \rangle$
 $\odot \langle -1 \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \rangle$

$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

where j starts with 0

- **Circular convolution** $\otimes$

$$\begin{array}{cccccccc} \langle & +1 & +1 & +1 & -1 & -1 & -1 & -1 & \rangle \\ \otimes & \langle & -1 & -1 & +1 & -1 & -1 & +1 & +1 & \rangle \\ \hline & \langle & & & & & & & & \rangle \end{array}$$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

- **Circular correlation** 

$\langle +1 \quad +1 \quad +1 \quad -1 \quad -1 \quad -1 \quad -1 \rangle$
 $\odot \langle -1 \quad -1 \quad +1 \quad -1 \quad -1 \quad +1 \quad +1 \rangle$

 $\langle -1 \quad -1 \quad -1 \quad -1 \quad +1 \quad +3 \quad -1 \rangle$

$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

where j starts with 0

- **Circular convolution** $\otimes$

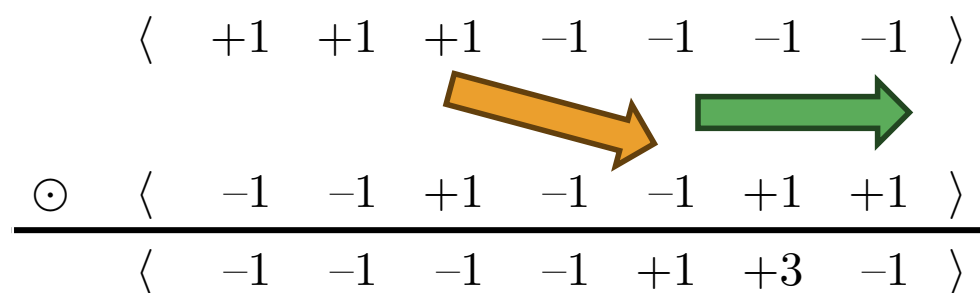
$$\begin{array}{c} \langle +1 +1 +1 -1 -1 -1 -1 \rangle \\ \otimes \langle -1 +1 +1 -1 -1 +1 -1 \rangle \\ \hline \langle \rangle \end{array}$$

Compute $\mathbf{y}^{\text{Rev}} = \langle y_0, y_{-1}, y_{-2}, \dots \rangle$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

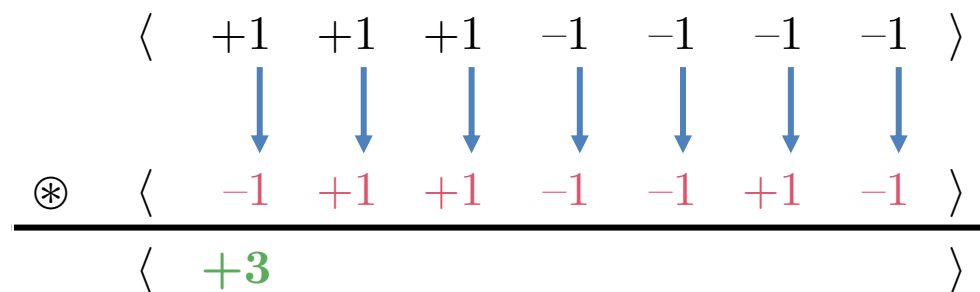
- **Circular correlation** $\odot$



$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

where j starts with 0

- **Circular convolution** $\otimes$



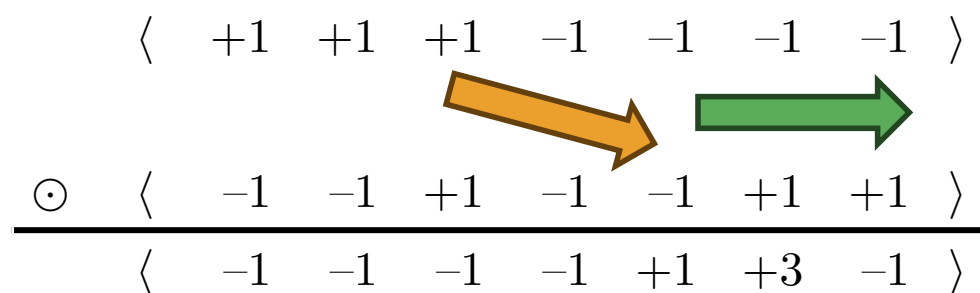
multiply then sum

$$\mathbf{y}^{\text{Rev}} = \langle y_0, y_{-1}, y_{-2}, \dots \rangle$$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

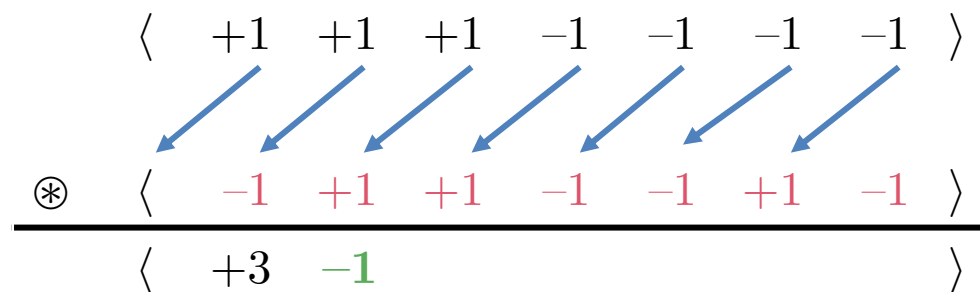
- **Circular correlation** $\odot$



$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

where j starts with 0

- **Circular convolution** $\otimes$



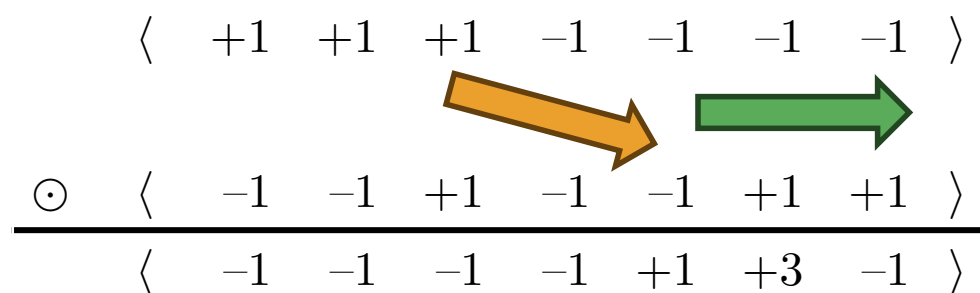
multiply then sum

$$\mathbf{y}^{\text{Rev}} = \langle y_0, y_{-1}, y_{-2}, \dots \rangle$$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

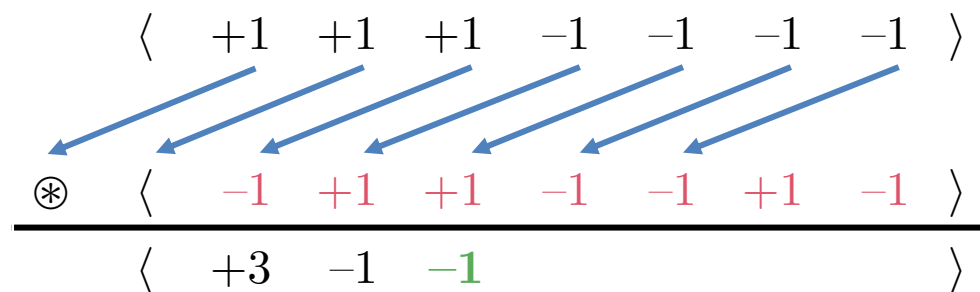
- **Circular correlation** $\odot$



$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

where j starts with 0

- **Circular convolution** $\otimes$



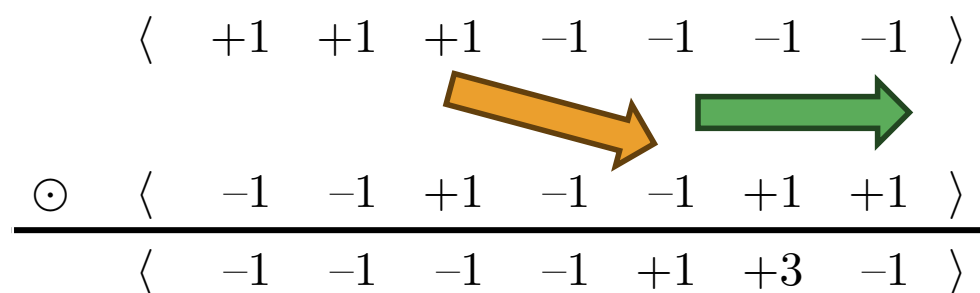
multiply then sum

$$\mathbf{y}^{\text{Rev}} = \langle y_0, y_{-1}, y_{-2}, \dots \rangle$$

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

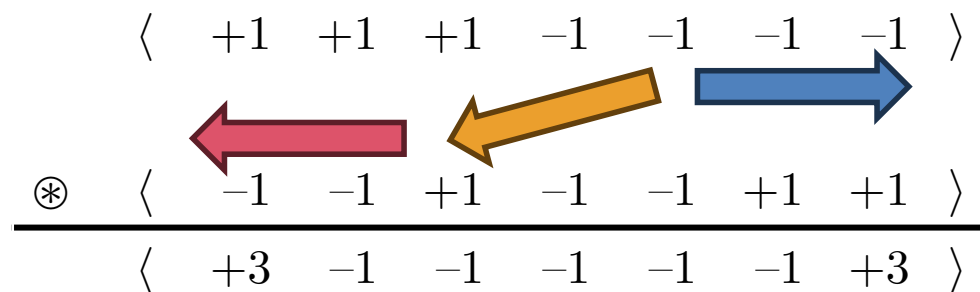
- **Circular correlation** $\odot$



$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

where j starts with 0

- **Circular convolution** $\otimes$



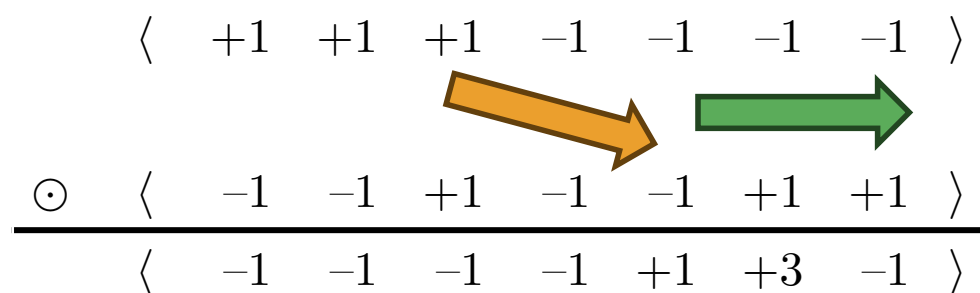
$$[\mathbf{x} \otimes \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^{-j} \mathbf{y}^{\text{Rev}})$$

where $\mathbf{y}^{\text{Rev}} = \langle y_0, y_{-1}, y_{-2}, \dots \rangle$
and DFT is Discrete Fourier Transform

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

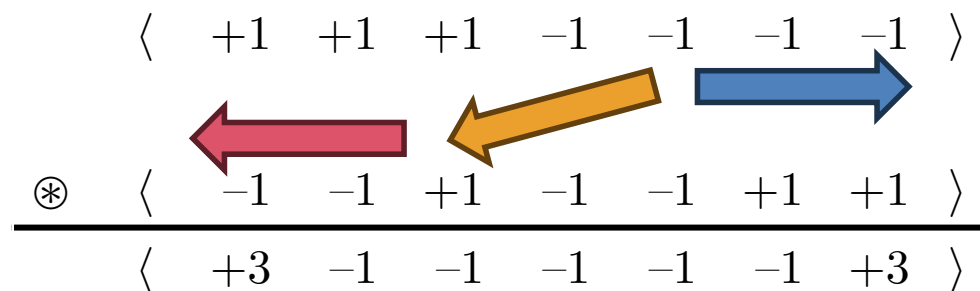
- **Circular correlation** $\odot$



$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

$$= \text{InvDFT}(\text{DFT}(\mathbf{x}) \otimes \overline{\text{DFT}(\mathbf{y})})$$

- **Circular convolution** $\otimes$



$$[\mathbf{x} \otimes \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^{-j} \mathbf{y}^{\text{Rev}})$$

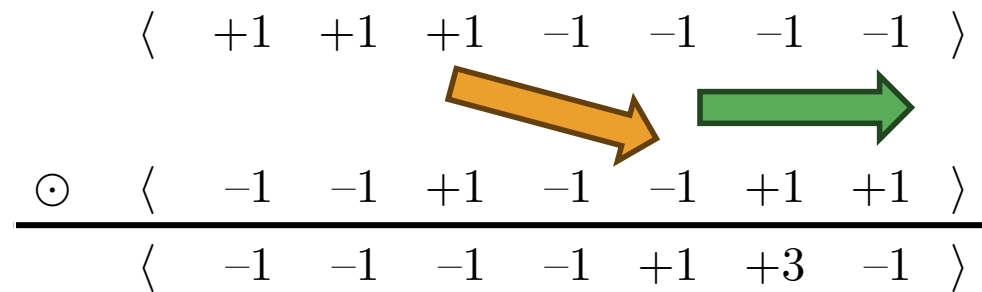
$$= \text{InvDFT}(\text{DFT}(\mathbf{x}) \otimes \text{DFT}(\mathbf{y}))$$

- **N.B.** DFT and InvDFT are computed with the $O(n \log n)$ time complexity

Type 2: Holographic Reduced Representation (HRR)

- **Non-commutative:** the order cannot change ($\mathbf{x} \odot \mathbf{y} \neq \mathbf{y} \odot \mathbf{x}$)

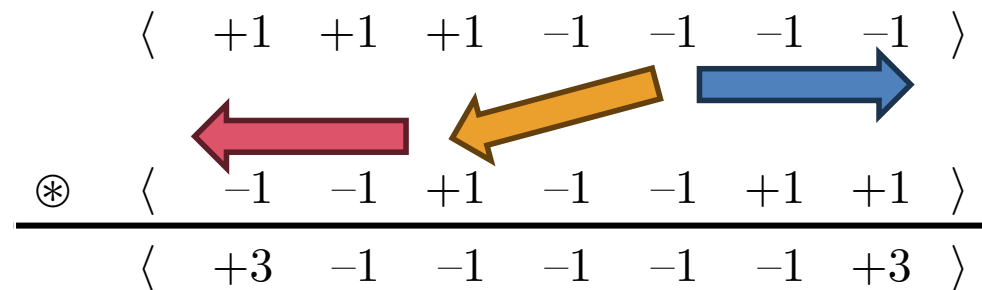
- **Circular correlation** $\odot$



$$[\mathbf{x} \odot \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^j \mathbf{y})$$

$$= \text{InvDFT}(\text{DFT}(\mathbf{x}) \otimes \overline{\text{DFT}(\mathbf{y})})$$

- **Circular convolution** $\otimes$



$$[\mathbf{x} \otimes \mathbf{y}]_j = \sum (\mathbf{x} \otimes p^{-j} \mathbf{y}^{\text{Rev}})$$

$$= \text{InvDFT}(\text{DFT}(\mathbf{x}) \otimes \text{DFT}(\mathbf{y}))$$

- **N.B.** Circular correlation and circular convolution are the inverse of each other

Encoding a Hash Table with HRR

- **Example 8:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$$x \circledast a$$

$$y \circledast b$$

$$z \circledast c$$

N.B. Releasing is non-commutative

3. Bundle all pairs with superposition.

$$h = (x \circledast a) \oplus (y \circledast b) \oplus (z \circledast c)$$

4. Release a value by multiplication.

$$(x \circledast a) \odot x = a \quad a \odot (x \circledast a) = x$$

5. Release a value by multiplication.

non-commutative (unswappable)

$$\begin{aligned} h \odot x &= [(x \circledast a) \odot x] \oplus \text{noise} \\ &\approx (x \circledast a) \odot x = a \end{aligned}$$

$$\begin{aligned} a \odot h &= [a \odot (x \circledast a)] \oplus \text{noise} \\ &\approx a \odot (x \circledast a) = x \end{aligned}$$

Type 3: Fourier Holographic Reduced Repr (FHRR)

- **Non-commutative:** the order can change ($\mathbf{x} \otimes \mathbf{y} = \mathbf{y} \otimes \mathbf{x}$)

Complex Numbers

$$a + bj = re^{i\theta}$$

$$\overline{a + bj} = a - bj$$

- **Elementwise addition** $\oplus$

$$\begin{array}{r} \langle \quad +1 \quad -j \quad +j \quad +1 \quad -1 \quad +1 \quad \rangle \\ \langle \quad +j \quad +1 \quad +j \quad -1 \quad -1 \quad +1 \quad \rangle \\ \oplus \quad \langle \quad +j \quad +j \quad +j \quad -j \quad -j \quad -j \quad \rangle \\ \hline \langle \quad 1+2j \quad +1 \quad +3j \quad -j \quad -2-j \quad 2-j \quad \rangle \end{array}$$

Then unitize each element

- **Elementwise multiplication** $\otimes$

$$\begin{array}{r} \langle \quad +1 \quad -j \quad +j \quad +1 \quad -1 \quad +1 \quad \rangle \\ \langle \quad +j \quad +1 \quad +j \quad -1 \quad -1 \quad +1 \quad \rangle \\ \otimes \quad \langle \quad +j \quad +j \quad +j \quad -j \quad -j \quad -j \quad \rangle \\ \hline \langle \quad -1 \quad +1 \quad -j \quad +j \quad -j \quad -j \quad \rangle \end{array}$$

N.B. For any HV $\mathbf{x}$,
 $\mathbf{x} \otimes \bar{\mathbf{x}} = 1$ always.

- **Permutation** p^N

$$\begin{array}{r} p^1 \quad \langle \quad \boxed{-j} \quad -1 \quad +j \quad -1 \quad -j \quad +1 \quad +1 \quad \boxed{} \quad \rangle \\ \hline \langle \quad \quad -1 \quad +j \quad -1 \quad -j \quad +1 \quad +1 \quad -j \quad \rangle \end{array}$$

T. A. Plate, [Holographic reduced representation: distributed representation for cognitive structures](#), Stanford: Center for the Study of Language and Information, 2003.

Encoding a Hash Table with FHRR

- **Example 9:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$$x \otimes a$$

$$y \otimes b$$

$$z \otimes c$$

N.B. Releasing is commutative

3. Bundle all pairs with superposition.

$$h = (x \otimes a) \oplus (y \otimes b) \oplus (z \otimes c)$$

4. Release a value by multiplication.

$$\bar{x} \otimes (x \otimes a) = a \quad (x \otimes a) \otimes \bar{x} = a$$

5. Release a value by multiplication.

commutative (swappable)

$$\begin{aligned} \bar{x} \otimes h &= [\bar{x} \otimes (x \otimes a)] \oplus [\bar{x} \otimes (y \otimes b)] \oplus [\bar{x} \otimes (z \otimes c)] \\ &= [\bar{x} \otimes (x \otimes a)] \oplus \text{noise} \oplus \text{noise} \\ &\approx \bar{x} \otimes (x \otimes a) = a \end{aligned}$$

Type 4: Bit Scatter Code (BSC)

P. Kanerva, [Fully distributed representation](#), In Proceedings of the Real World Computing Symposium. 358–365. 1997.

- **Commutative:** the order can change ($\mathbf{x} \otimes \mathbf{y} = \mathbf{y} \otimes \mathbf{x}$)

- **Majority rule** $\oplus$

$$\begin{array}{r}
 \langle \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad \rangle \\
 \langle \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad \rangle \\
 \oplus \quad \langle \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \rangle \\
 \hline
 \langle \quad 1 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 0 \quad \rangle
 \end{array}$$

- **Elementwise XOR** $\otimes$

$$\begin{array}{r}
 \langle \quad 1 \quad 0 \quad 1 \quad 1 \quad 0 \quad 1 \quad 0 \quad \rangle \\
 \langle \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 0 \quad 0 \quad \rangle \\
 \otimes \quad \langle \quad 0 \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \rangle \\
 \hline
 \langle \quad 0 \quad 1 \quad 1 \quad 1 \quad 0 \quad 0 \quad 1 \quad \rangle
 \end{array}$$

N.B. For any HV $\mathbf{x}$,
 $\mathbf{x} \otimes \bar{\mathbf{x}} = 1$ always.

- **Permutation** p^N

$$\begin{array}{r}
 p^1 \quad \langle \quad \boxed{0} \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad \boxed{} \quad \rangle \\
 \hline
 \langle \quad \quad 0 \quad 1 \quad 0 \quad 0 \quad 1 \quad 1 \quad 0 \quad \rangle
 \end{array}$$

Encoding a Hash Table with BSC

- **Example 10:** hash table $h = \{ 'x' : 'a', 'y' : 'b', 'z' : 'c' \}$

1. Initialize with random vectors.

Key vectors: x, y, z

Value vectors: a, b, c

2. Bind each pair of key and value.

$$x \otimes a$$

$$y \otimes b$$

$$z \otimes c$$

N.B. Releasing is commutative

3. Bundle all pairs with superposition.

$$h = (x \otimes a) \oplus (y \otimes b) \oplus (z \otimes c)$$

4. Release a value by multiplication.

$$\bar{x} \otimes (x \otimes a) = a \quad (x \otimes a) \otimes \bar{x} = a$$

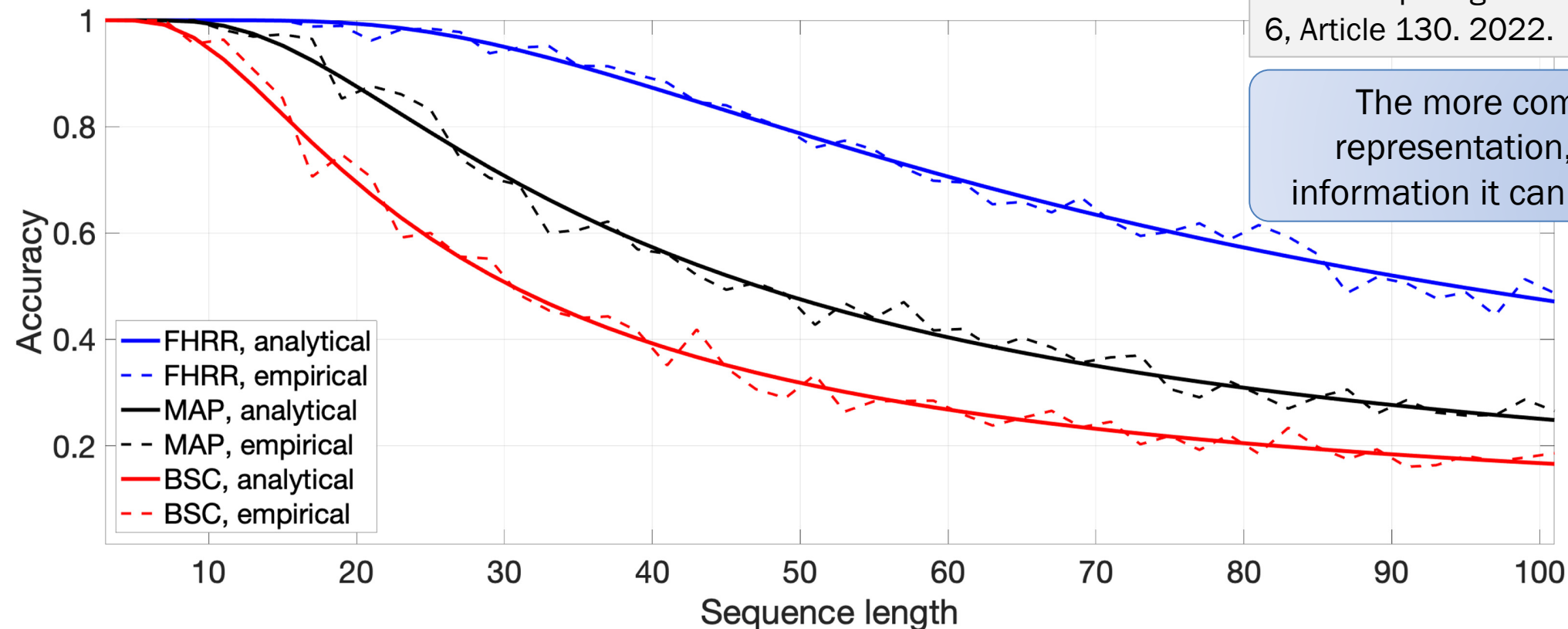
5. Release a value by multiplication.

commutative (swappable)

$$\begin{aligned} \bar{x} \otimes h &= [\bar{x} \otimes (x \otimes a)] \oplus [\bar{x} \otimes (y \otimes b)] \oplus [\bar{x} \otimes (z \otimes c)] \\ &= [\bar{x} \otimes (x \otimes a)] \oplus \text{noise} \oplus \text{noise} \\ &\approx \bar{x} \otimes (x \otimes a) = a \end{aligned}$$

Which Architecture Then?

D. Kleyko et al., [A Survey on Hyperdimensional Computing aka Vector Symbolic Architectures, Part I: Models and Data Transformations](#), ACM Computing Surveys, Vol. 55, No. 6, Article 130. 2022.



The more complex the representation, the more information it can encapsulate

Fig. 2. The analytical and empirical accuracies of recovering sequences from HVs against the sequence length for three HDC/VSA models, $D = 256$, $N = 64$. The reported empirical accuracies were averaged over 5 simulations with randomly initialized item memories. [D = dimensions, N = items]

5. Implementation with TorchHD

<https://tinyurl.com/4ea28dt7>

TorchHD for MAP

```
import torch as T
import torchhd as H

dimensions = 10_000      # 10,000 dimensions
vsa = 'MAP'
keys = H.random(10, dimensions, vsa=vsa)      # Keys: 10 symbols
values = H.random(20, dimensions, vsa=vsa)    # Values: 20 symbols

pair1 = keys[0].bind(values[4])                # pair1 = (0,4)
pair2 = keys[7].bind(values[15])              # pair2 = (7,15)
pair3 = keys[2].bind(values[9])               # pair3 = (2,9)

hash_table = pair1.bundle(pair2).bundle(pair3)

vec = hash_table.bind(keys[7])                # Retrieve the value of key 7

sim_matrix = values.cosine_similarity(vec)     # Find its symbol
value = sim_matrix.argmax()
```

- HDC library based on PyTorch with GPU capability
- HV algebra
 - `vec1.bind(vec2)`
 - `vec1.bundle(vec2)`
 - `vec1.permute(cnt)`
- Associative memory
 - `mat.cosine_similarity(vec)`
 - `mat.hamming_similarity(vec)`

TorchHD for HRR

```
import torch as T
import torchhd as H

dimensions = 10_000
vsa = 'HRR'          # Other choices are 'BSC' and 'FHRR'
keys = H.random(10, dimensions, vsa=vsa)
values = H.random(20, dimensions, vsa=vsa)

pair1 = keys[0].bind(values[4])      # Circular convolution
pair2 = keys[7].bind(values[15])
pair3 = keys[2].bind(values[9])

hash_table = pair1.bundle(pair2).bundle(pair3)

vec = hash_table.bind(keys[7].inverse()) # Unbinding

sim_matrix = values.dot_similarity(vec)  # Dot similarity
value = sim_matrix.argmax()
```

- HRR in TorchHD
 - Circular convolution is commutative (!)
 - Releasing is done by binding with HV inverse
- HV algebra
 - `vec1.bind(vec2)`
 - `vec1.bundle(vec2)`
 - `vec1.permute(cnt)`
 - `vec1.inverse()`
- Associative memory
 - `mat.dot_similarity(vec)`

6. NLP and Knowledge Graph via HDC

Language Recognition (Joshi et al., QI-2016)

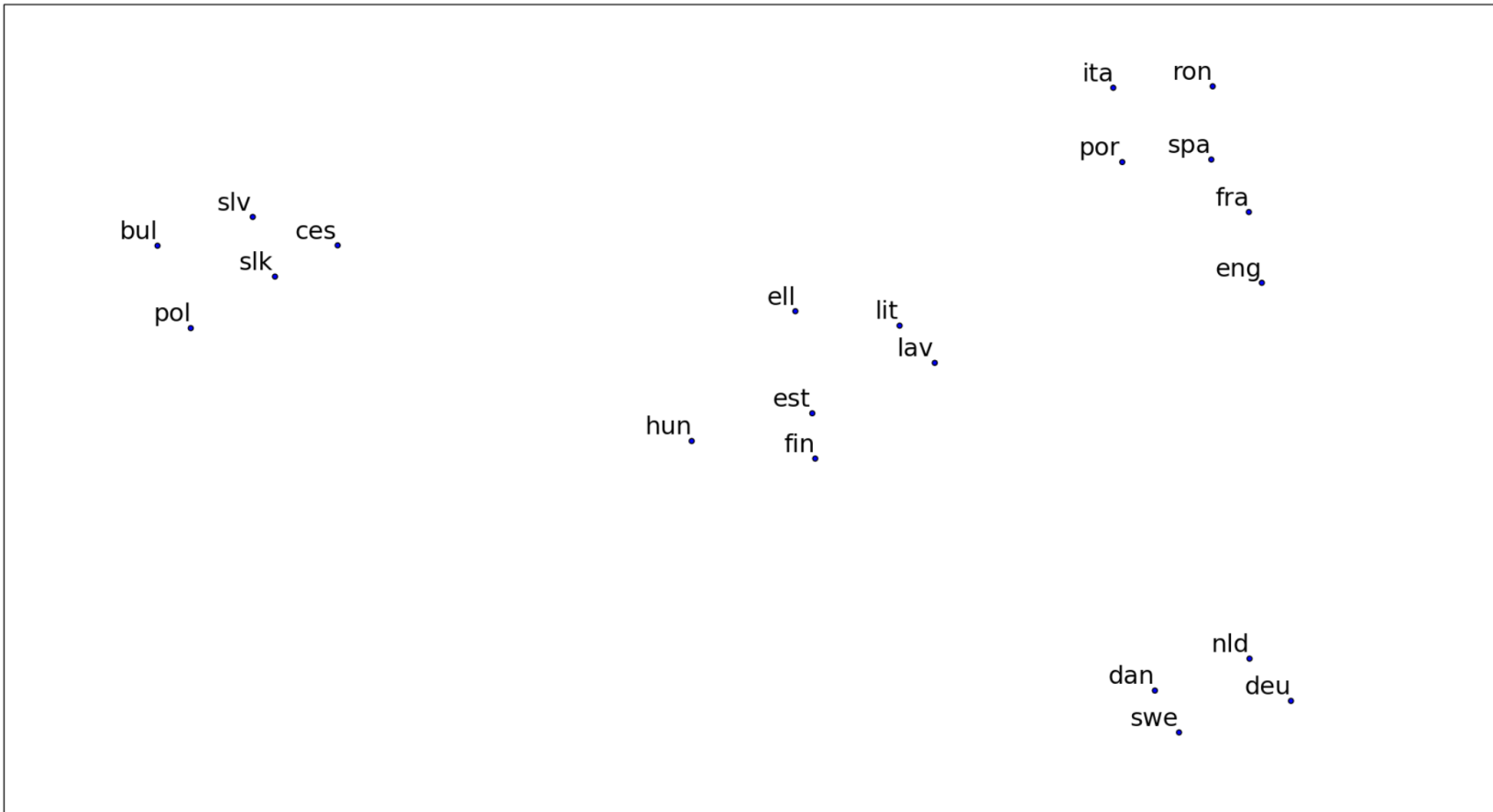
- Identifying the language of a given text with MAP
 - Romanize all non-Roman texts (Greek, Russian, etc.). $\Psi\text{ITTAKO}\Sigma \Rightarrow \text{'PSITTAKOS'}$
 - Initialize each character with random vectors. $\mathbf{a} \ \mathbf{b} \ \mathbf{c} \ \mathbf{d} \ \mathbf{e} \ \dots \ \mathbf{x} \ \mathbf{y} \ \mathbf{z} \ _$
 - Encode trigrams with permutation and binding. $|\text{MIS}\rangle = p^2 \mathbf{m} \otimes p^1 \mathbf{i} \otimes \mathbf{s}$
 - Extract all trigrams by sliding window.

$$\begin{aligned} &\text{'I AM GROOT'} \\ \Rightarrow &\text{I_A, _AM, AM_ , M_G, _GR, GRO, ROO, OOT} \end{aligned}$$
 - Bundle all trigrams of the string.

$$\begin{aligned} &|\text{I AM GROOT}\rangle \\ = &|\text{I_A}\rangle \oplus |\text{_AM}\rangle \oplus |\text{AM_}\rangle \oplus |\text{M_G}\rangle \\ &\oplus |\text{_GR}\rangle \oplus |\text{GRO}\rangle \oplus |\text{ROO}\rangle \oplus |\text{OOT}\rangle \end{aligned}$$
 - Identify the language of the input text by comparing its HV with the HVs of each language.

$$L^* = \arg \max_L \text{cossim}(|\text{string}\rangle, |L\rangle)$$

Language Recognition (Joshi et al., QI-2016)



- Distribution of language HVs
 - Aligned with language families
 - Oriented by spelling and romanization

N	Accuracy
1	74.9
2	94.0
3	97.3
4	97.8
5	97.3

Joshi, A., Halseth, J., and Kanerva, P. (2017). [Language geometry using random indexing](#). In J. A. de Barros, B. Coecke & E. Pothos (eds.) Quantum Interaction, 10th International Conference, QI 2016, pp. 265-274. Springer.

Semantics (Karlgrén and Kanerva, NLE-2019)

- Lexical semantics in dynamic data stream

1. Encode each word with permutation.
2. Encode a raw text by weighted bundling.
 - The weight is equivalent to TF·IDF, but it can be dynamically computed from the data stream
3. Encode an input string annotated with semantic roles by permutation.
 [That **seagull**]_{ACTOR} ate [my **sandwiches**]_{ACTEE}
4. Encode each tagged trigram
 $(w_1/t_1, w_2/t_2, w_3/t_3)$ by permutation.
5. Encode a tagged text with bundling.

J. Karlgrén and P. Kanerva. [High-dimensional distributed semantic spaces for utterances](#). Natural Language Engineering 25, no. 4 (2019): 503-517.

$$|\text{under}\rangle = p^4 \mathbf{u} \otimes p^3 \mathbf{n} \otimes p^2 \mathbf{d} \otimes p^1 \mathbf{e} \otimes \mathbf{r}$$

$$|w_1 \dots w_N\rangle_{\text{raw}} = \bigoplus_{k=1}^N \left[|w_k\rangle \cdot e^{-\lambda \cdot \frac{\#(w_k)}{V}} \right]$$

where λ is the aggressive factor, $\#(w_k)$ is the frequency, and V is the size of learned vocabulary

$$|S\rangle_{\text{roles}} = \bigoplus_{r=1}^R p^{(r)} |h_r\rangle$$

where $p^{(r)}$ is permutation specific to role r with head h_r

$$|w_1/t_1, w_2/t_2, w_3/t_3\rangle = p^{(t_1)} p^{(t_2)} p^{(t_3)} |w_1 w_2 w_3\rangle_{\text{raw}}$$

where each $p^{(t)}$ is permutation specific to tag t

$$|S\rangle_{\text{total}} = |S\rangle_{\text{raw}} \oplus |S\rangle_{\text{roles}} \oplus \bigoplus_{\text{trigram}} |\text{trigram}\rangle$$

Transformers + VSA

I. Schlag, P. Smolensky, et al. (2019). [Enhancing the Transformer With Explicit Relational Encoding for Math Problem Solving](#). Workshop on Context and Compositionality in Biological and Artificial Neural Systems (NeurIPS 2019).

- Enabling Transformer models to solve math problems with VSA-based explicit relation encoding

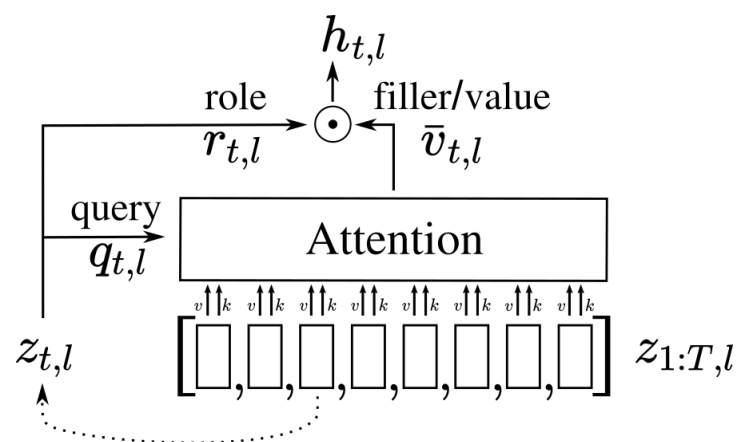


Figure 1: A simplified illustration of our TP-Attention mechanism for one head at position t in layer l . The main difference from standard Attention is the additional role representation that is element-wise multiplied with the filler/value representation.

	weights	steps	train	interpolation acc	>95%	extrapolation acc	>95%
Simple LSTM	18M	500k	-	57.00%	6	41.00%	1
Transformer (Saxton et al.)	30M	500k	-	76.00%	13	50.00%	1
Transformer (ours)	44.2M	500k	83.06%	75.33%	12	52.42%	1
		700k	85.01%	77.42%	14	52.00%	2
TP-Transformer (ours)	49.2M	500k	85.41%	78.30%	16	52.22%	2
		700k	87.25%	80.67%	18	52.48%	3
		1.7M	91.04%	84.24%	25	55.40%	3

(- 5 7 2) / 1 5 4 * 4 5 / 1 8 - ((- 1 0 4) / (- 2 8) + - 4) %
2 1 3 10 4 9 1 16 13 4 6 11 10 9 16 13 6 0 8 0 0 1 8 10 4 9 1 16 1 19 13 6 1 0 8 0 15 9 8 2

W h a t i s ((- 9 2) / (- 1 3 8 0)) / (1 2 6 / (- 1 3 5)) ? &
2 7 18 5 12 17 18 5 8 7 7 8 4 9 1 16 7 3 13 4 6 6 7 7 16 11 13 4 6 11 7 3 10 4 9 7 7 17 2

W h a t i s (1 0 + (- 2 2 5) / 3 6) / (4 2 / 1 6 8) ? %
2 1 18 5 12 17 18 5 15 7 10 9 0 8 0 1 19 10 4 9 1 16 13 6 11 16 11 13 6 11 10 4 9 7 17 2

W h a t i s ((- 9 2) / (- 1 3 8 0)) / (1 2 6 / (- 1 3 5)) ? %
2 7 18 5 12 17 18 5 8 7 7 8 4 9 1 16 7 3 13 4 6 6 7 7 16 11 13 4 6 11 7 3 10 4 9 7 7 17 2

[N] = cluster ID of embedding vector
1 / 2 0 + 2 0 3 2 8 / (- 8 9 7 6 0) %
2 9 16 13 6 15 8 0 10 4 4 4 9 16 1 19 13 4 4 6 6 1 2

Knowledge Graph (Poduval et al., 2022)

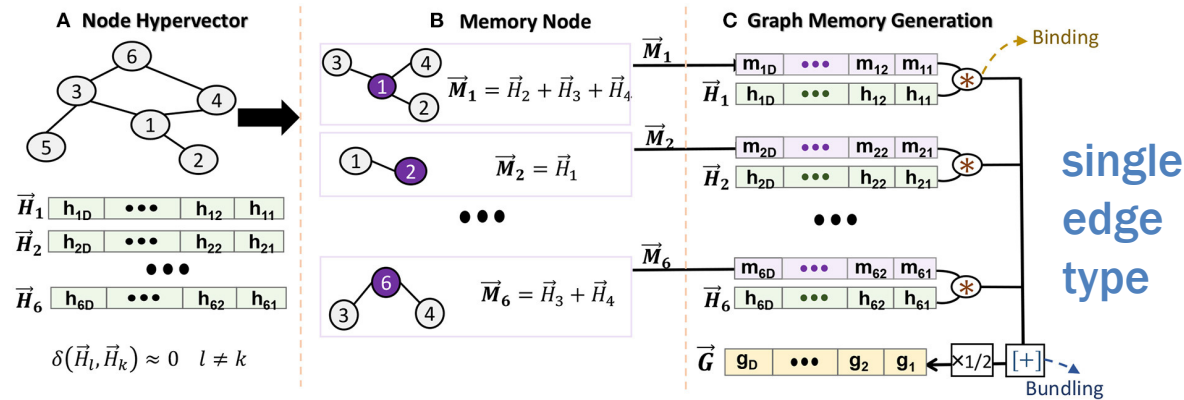


FIGURE 2 | Graph memory encoding in GraphHD: (A) node hypervector generation, (B) creating a node memory, and (C) graph memory generation.

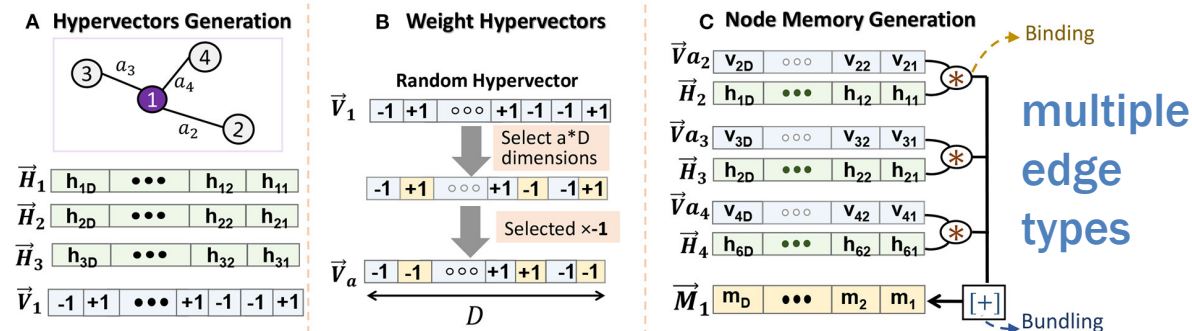
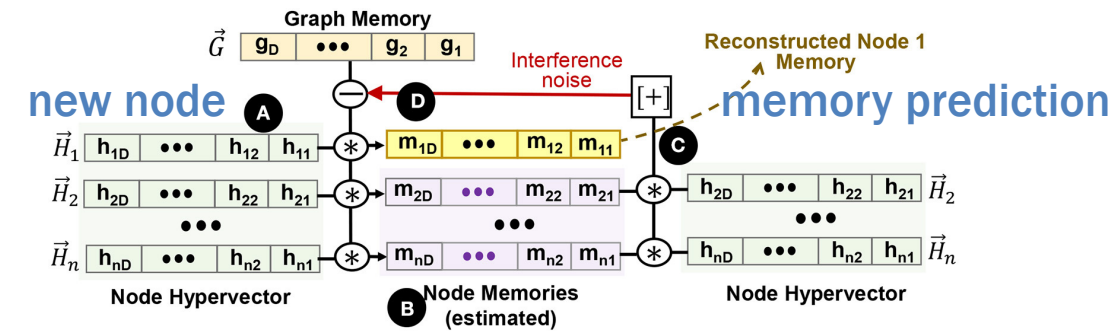
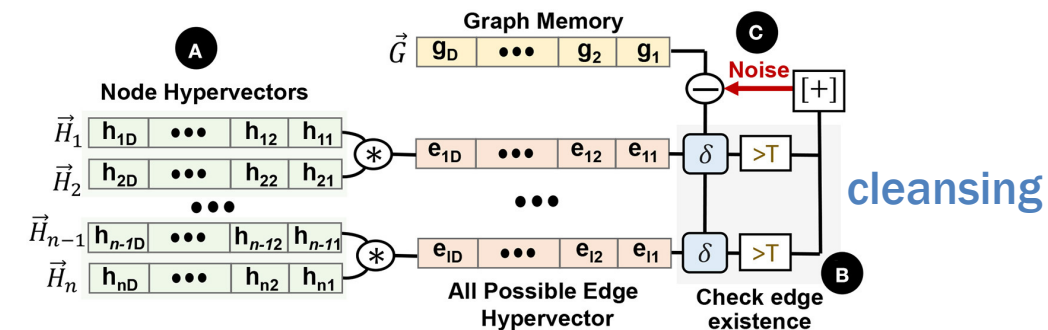


FIGURE 3 | Node memory encoding in GraphHD for weighted graph: (A) node hypervector generation, (B) creating weight hypervectors, and (C) node memory generation.

- Node memory reconstruction



- Graph memory reconstruction



- Graph matching via graph memory

HyperSeg (Park et al., InterSpeech-2023)

- Unsupervised topic segmentation with MAP
 1. Eliminate all stop words.
 2. Initialize each word with random vectors.
 3. For each text chunk of length L , permute each word HV by its position i
 4. Compute the HV of the chunk by bundling and soft quantization (i.e. Softmax)
 5. For every neighboring text chunks $\mathbf{u}_i$ and $\mathbf{u}_{i+1}$, compute the segmentation score
 6. Compute the threshold of segmentation θ from the mean and standard deviation of the segmentation scores: μ_S and σ_S

$$\mathbf{w}_i^{\text{pos}} = p^{L-i-1} \mathbf{w}_i$$

$$\mathbf{u} = \text{Softmax} \left[\bigoplus_{i=1}^L \mathbf{w}_i^{\text{pos}} \right]$$

$$S = \frac{\mathbf{u}_i \cdot \mathbf{u}_{i+1}}{|\mathbf{u}_i| \times |\mathbf{u}_{i+1}|}$$

$$\theta = \mu_S - \sigma_S$$

Two chunks are separated if their score $S < \theta$

HyperSeg (Park et al., InterSpeech-2023)

Table 2: Transcript segmentation results. HyperSeg performs best even without the ground-truth segment count provided to baselines. "BS" stands for boundary similarity. Results are averaged over 5 runs with different random seeds. Higher is better for both metrics.

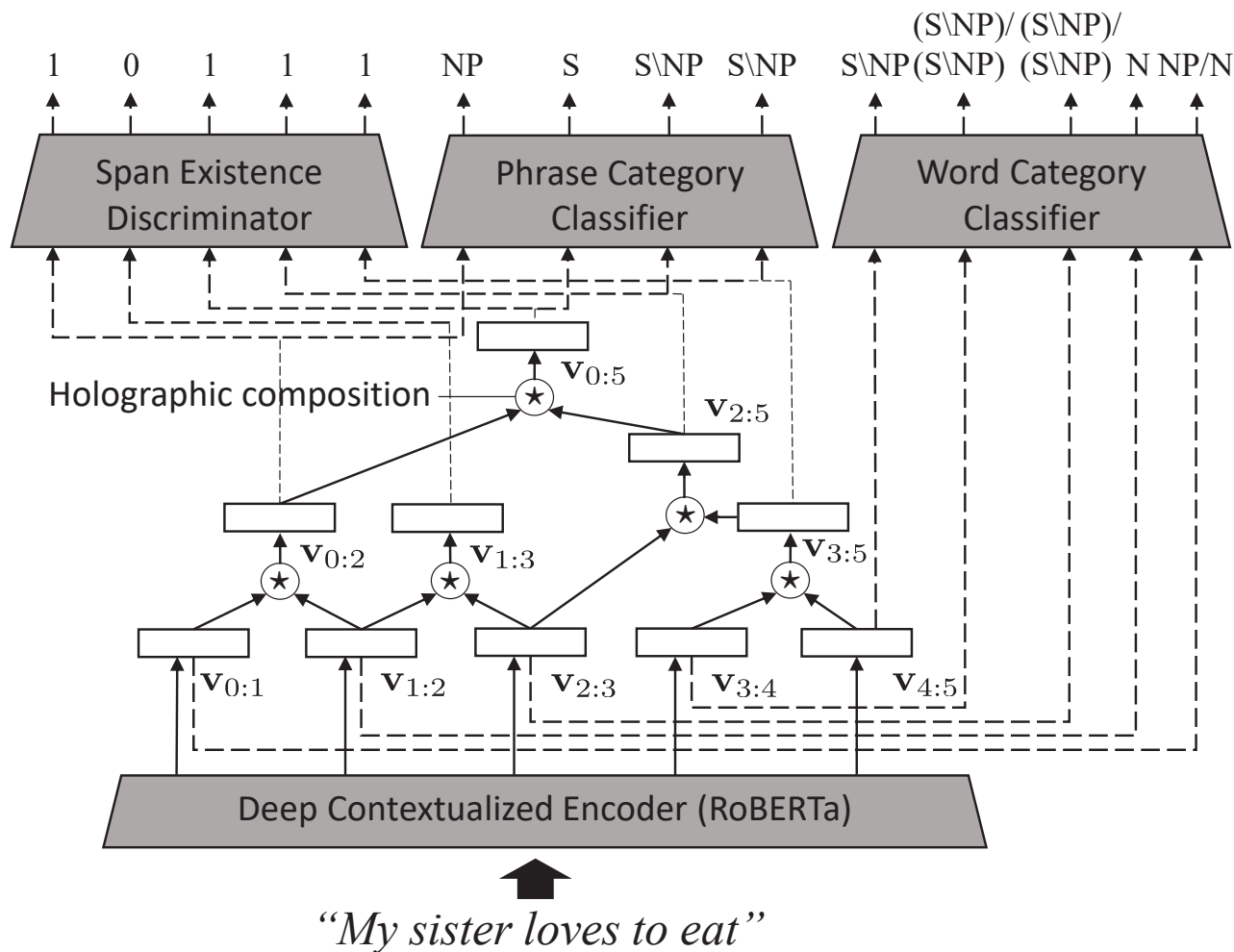
	AMI		ICSI		Doc2Dial		VT-SSum		Wiki-727k	
Model	F1	BS	F1	BS	F1	BS	F1	BS	F1	BS
Random	5.19	2.75	5.09	2.64	10.76	6.89	23.91	13.30	28.37	16.60
GraphSeg [17]	4.22	2.22	2.75	1.43	10.06	5.35	2.28	1.18	21.09	12.26
BERTSeg [13]	4.43	2.36	1.58	0.81	2.95	1.56	8.84	4.75	17.91	10.23
CohereSeg [14]	8.77	4.63	7.79	4.03	16.58	10.09	25.98	14.57	28.41	17.60
HyperSeg (ours)	9.41	4.93	6.94	3.55	18.49	10.54	27.35	15.15	29.82	17.45

Table 3: Average milliseconds elapsed per utterance. HyperSeg is 10 times faster than the next fastest baseline.

Model	AMI	ICSI	Doc2Dial	VT-SSum	Wiki-727k	Mean	× HyperSeg
GraphSeg [17]	11.60	13.71	10.66	11.11	9.73	11.36	× 10.42
BERTSeg [13]	11.92	12.02	12.82	11.45	11.22	11.89	× 10.91
CohereSeg [14]	15.99	14.75	15.15	15.28	15.76	15.39	× 14.12
HyperSeg (ours)	0.77	0.70	1.19	1.28	1.49	1.09	× 1

Park, S., Seo, J., Lee, J. (2023). [Unsupervised Dialogue Topic Segmentation in Hyperdimensional Space](#). Proc. INTERSPEECH 2023, 730-734. **GitHub** ⇒ <https://github.com/seongminp/hyperseg>

Holographic CCG Parsing (Yamaki et al., ACL-2023)



- **Super-Tagging** (Clark and Curran, 2007) + **Recursive NNs** (Socher, 2013)
- 1. Initialize each word with LLM (e.g. RoBERTa).
- 2. Predict word categories with their HVs.
- 3. Compute phrase HVs by **non-commutative** binding ($\otimes$).
- 4. Predict span existence and phrase categories by their HVs.
- 5. Recursively compute node HVs until we reach the root node.

Holographic CCG Parsing (Yamaki et al., ACL-2023)

Model	Super-Tagger	Parser	Acc	LF
Lewis et al. (2016)	LSTM	A*	94.7	88.1
Vaswani et al. (2016)	LSTM	C&C	94.5	88.32
Yoshikawa et al. (2017)	LSTM	A* (LSTM)	–	88.8
Stanojević and Steedman (2020)	LSTM	Shift-Reduce (LSTM)	–	90.6
Tian et al. (2020)	Attentive-GCNN	EasyCCG	96.25	90.58
Bhargava and Penn (2020)	LSTM decoder	C&C	96.00	90.9
Liu et al. (2021)	Category Generator	C&C	96.05	90.87
Prange et al. (2021)	Tree-Structured decoder	C&C	96.22	90.91
Kogkalidis and Moortgat (2022)	Heterogeneous Dynamic Convolutions	–	96.29	–
Clark (2021)	Tian et al. (2020)	C&C	–	91.9
		Span-based	–	92.9
Ours ($\mathcal{L}_w + \mathcal{L}_p + \mathcal{L}_s$, Real)	Holographic	C&C	96.60	92.12
		Span-based	–	92.67

Table 2: Comparison of the proposed model and existing methods; best results are shown in bold.

Yamaki, R., Taniguchi, T., and Mochihashi, D. (2023). [Holographic CCG Parsing](#). Proceedings of Association for Computational Linguistics, ACL-2023.

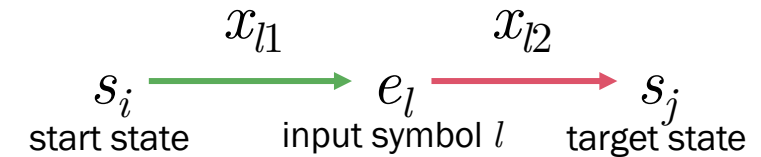
Attractor Neural Networks

M. Cotteret, H. Greatedorex, et al. [Vector Symbolic Finite State Machines in Attractor Neural Networks](#). Neural Computation 36(4). 2024.

- Robust FSA using Hopfield networks (Hopfield, 1982)
 - States (attractors) and input symbols (stimuli) can be noisy
 - **Motivation:** Hopfield network $\mathbf{W}$ is used for recovering a noisy vector $\hat{\mathbf{x}}$ to one of the stored HVs $\{\mathbf{x}_1, \dots, \mathbf{x}_K\}$ by simple matrix multiplication $\mathbf{x}_{k^*} = \mathbf{W}\hat{\mathbf{x}}$, where $K < 0.14D$
 - State transition is divided into two steps: **(1) start state $\rightarrow$ symbol** and **(2) symbol $\rightarrow$ target state**, to prevent state skipping and oscillations between states due to noises
 - **Attractor network:** $\mathbf{W} = \mathbf{W}_{\text{state}} + \mathbf{W}_{\text{symbol}}$
 - State transition is derived by (s_i, l) as follows

$$\mathbf{e} \approx \mathbf{W}(\mathbf{s}_i \otimes H(\mathbf{x}_{l1}))$$

$$\mathbf{s}_j \approx \mathbf{W}(\mathbf{e} \otimes H(\mathbf{x}_{l2}))$$



$$\mathbf{W}_{\text{state}} = \frac{1}{D} \sum_{k=1}^K \mathbf{s}_k \mathbf{s}_k^\top$$

$$\mathbf{W}_{\text{symbol}} = \frac{1}{D} \sum_{k=1}^Q \mathbf{E}^{(k)}$$

where:

$$\begin{aligned} \mathbf{E}^{(l)} = & \mathbf{e}_l \mathbf{e}_l^\top \\ & + H(\mathbf{x}_{l1}) \otimes (\mathbf{e}_l - \mathbf{s}_i) (\mathbf{s}_i \otimes \mathbf{x}_{l1})^\top \\ & + H(\mathbf{x}_{l2}) \otimes (\mathbf{s}_j - \mathbf{e}_l) (\mathbf{e}_l \otimes \mathbf{x}_{l2})^\top \end{aligned}$$

and $H(\mathbf{x})$ is Heaviside step function

7. Conclusion

Conclusion

- Hyperdimensional computing (HDC)
 - Non-von Neumann Architecture
 - **HV algebra**: random, bind, bundle, permute, and similarity
 - Vector representation of complex data structures
- Vector-symbolic architecture (VSA)
 - Bridge between symbolic logic and vector-based machine learning techniques
 - Accurate, noise-tolerant vector operations for symbolic computation and logical inference
 - Popular VSAs: MAP, HRR, FHRR, and BSC
- ML, NLP, and knowledge graph in HDC

Major Differences

- **ANN**: Dense vectors manifest **similarity** for ease of analogy to the experiences
- **HDC**: HVs manifest **dissimilarity**, which makes each symbol unique and noise-tolerant

Thank You

prachya.boonkwan@nectec.or.th

kaamanita@gmail.com